项目6 UrbanPipe-MLLM：真实场景下城市管道巡检视频的多模态异常检测研究

- 视频：

- 第一章：**多模态基础** - 第二、三章：**链接 1（对应到 2.6.4）；讲解2（多模态到结束）** - 第四章：**面经讲解**

代码复制等：Urbanpipe（可复制）

一、基础知识

> **讲解视频：多模态基础**

1.1、视觉 Transformer

> 论文来源：Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, 2020

> 链接：https://arxiv.org/pdf/2010.11929

1.1.1、从卷积到 Transformer

在过去十余年中，卷积神经网络（Convolutional Neural Networks， CNNs）在计算机视觉领域取得了革命性的成功。然而，随着视觉任务复杂度的提升，卷积范式逐渐暴露出若干内在局限性，而 Transformer 架构正是为解决这些问题而生。

1. 并非所有像素同等重要

卷积操作对图像中的所有像素一视同仁，使用同一组滤波器处理每一个局部区域。然而，在诸如目标检测、图像分类等任务中，**前景区域通常远比背景区域重要。**卷积机制无法显式地区分这些差异。

2. 难以建模远距离依赖关系

卷积核的感受野通常局限于局部邻域，尽管可以通过增大卷积核尺寸或加深网络层数来扩大感受野，但这会显著增加模型参数量和计算成本，且仍难以有效捕捉**空间上相距较远的语义关系。**

1.1.2、视觉 Transformer 的提出

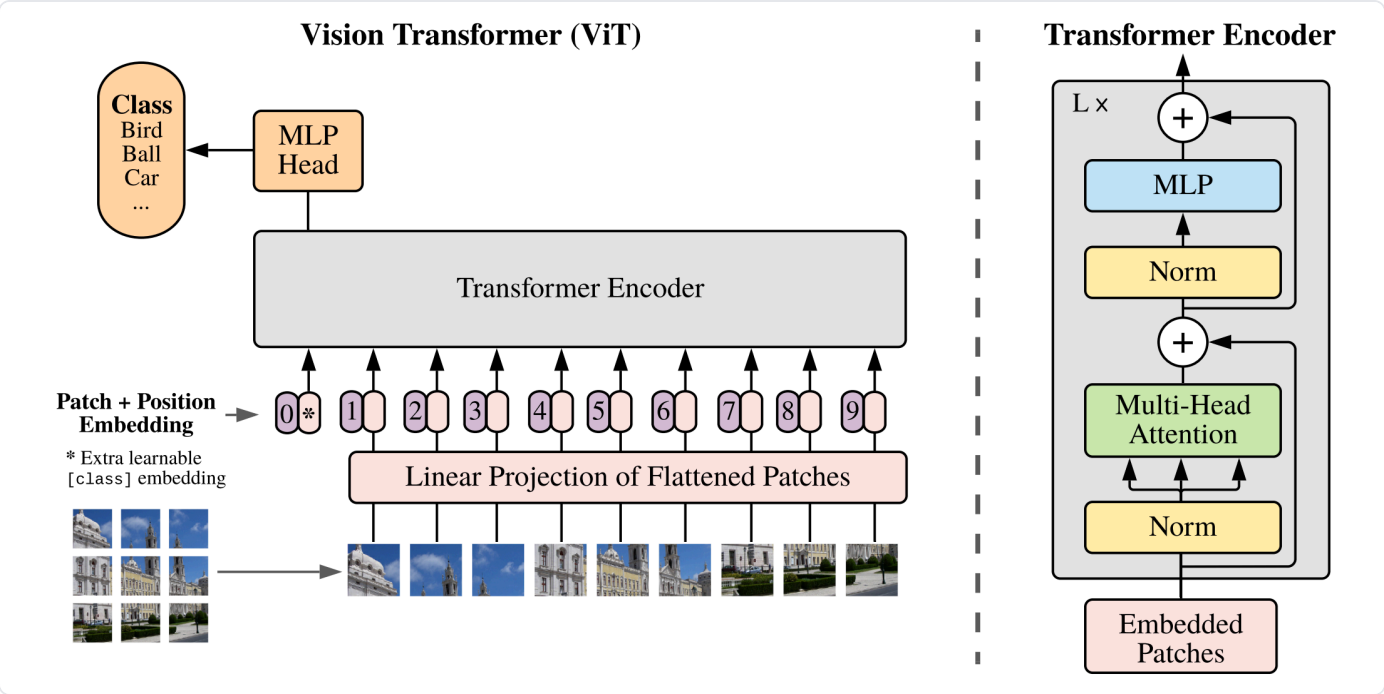
视觉 Transformer（Vision Transformer， 简称 ViT）是一种将**自注意力机制**引入图像建模的架构，标志着视觉建模范式从“局部感知 + 层级堆叠”向“全局建模 + 序列建模”的转变。

ViT 的设计思想源于自然语言处理领域中 Transformer 的成功应用。Transformer 最初由 Vaswani 等人在 2017 年提出（论文《Attention Is All You Need》），是一种专门用于处理序列数据的神经网络架构。

ViT 的核心在于：**将图像视为由一系列“视觉词元（visual tokens）”组成的序列，并使用标准 Transformer 编码器进行建模。**

1.1.3、将 Transformer 应用于图像：基本思想

由于图像是二维结构，而 Transformer 擅长处理一维序列，因此 ViT 的首要任务是：**将图像转换为可供 Transformer 处理的序列表示。**



ViT 将一张图像划分为固定大小的图像块（patches），对每个图像块进行线性映射，再加上位置嵌入（position embeddings），并将得到的向量序列输入到标准的 Transformer 编码器中。

这一过程主要包括以下步骤：

1. 图像块划分

给定一张大小为 $（H \times W \times C）$ 的输入图像，将其划分为若干个固定大小为 $（P \times P）$ 的图像块（patch）。这样，图像将被分割为：$N = \frac{HW}{P^2}$ 个图像块。每个图像块对应图像中的一个局部区域，类似于 NLP 中的“词”。

2 图像块展平

每个图像块包含 $（P \times P \times C）$ 个像素值，将其按通道展开为一个一维向量：$\mathbf{x}_i \in \mathbb{R}^{P^2C}$。从而使模型能够将每个图像块视为序列中的一个元素。

（b， c， H， W） → （b， patch 数， 每个 patch 的像素数×通道数）

3. 线性映射

为了将高维图像块向量映射到模型的隐藏维度空间中，每个图像块向量通过一个可训练的线性变换：

$$\mathbf{z}_i = \mathbf{W}\mathbf{x}_i + \mathbf{b}, \quad \mathbf{z}_i \in \mathbb{R}^D$$

其中 （D） 为 Transformer 的嵌入维度。

该过程等价于对图像块进行一次线性编码，使其成为可供 Transformer 处理的“视觉词向量”。

---

4. 位置编码

Transformer 本身不具备对序列顺序的感知能力，因此必须为每个图像块嵌入添加位置编码，以保留其空间位置信息：

$$\mathbf{z}_i' = \mathbf{z}_i + \mathbf{p}_i$$

其中 （ $\mathbf{p}_i$） 表示第 （i） 个图像块的位置嵌入。

位置编码使模型能够理解不同图像块之间的空间关系，例如上下、左右、远近等结构信息。

---

5. 分类标记

为了实现图像级分类任务，在图像块嵌入序列的最前端加入一个特殊的、可学习的向量：

$$\mathbf{z}_0 = \mathbf{x}_{\text{class}}$$

该向量称为**分类标记（[CLS] token）**。在 Transformer 编码器输出端，该标记对应的状态向量被视为整幅图像的全局表示，并用于最终分类预测。

---

6. 输入 Transformer 编码器

最终输入到 Transformer 编码器的序列形式为：

$$\mathbf{Z}_0 = [\mathbf{x}_{\text{class}}, \mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_N] + \mathbf{P}$$

该序列随后被送入由多层堆叠组成的 Transformer 编码器，每一层包含以下两个核心模块：

1. **多头自注意力机制**

用于建模任意图像块之间的全局依赖关系，使模型能够自动关注对当前任务最重要的区域。

1. **前馈神经网络模块**

对每个位置的表示进行非线性变换，增强特征表达能力。

在每个模块前，均应用层归一化（Layer Normalization， LN），并采用残差连接以提高训练稳定性与收敛速度。

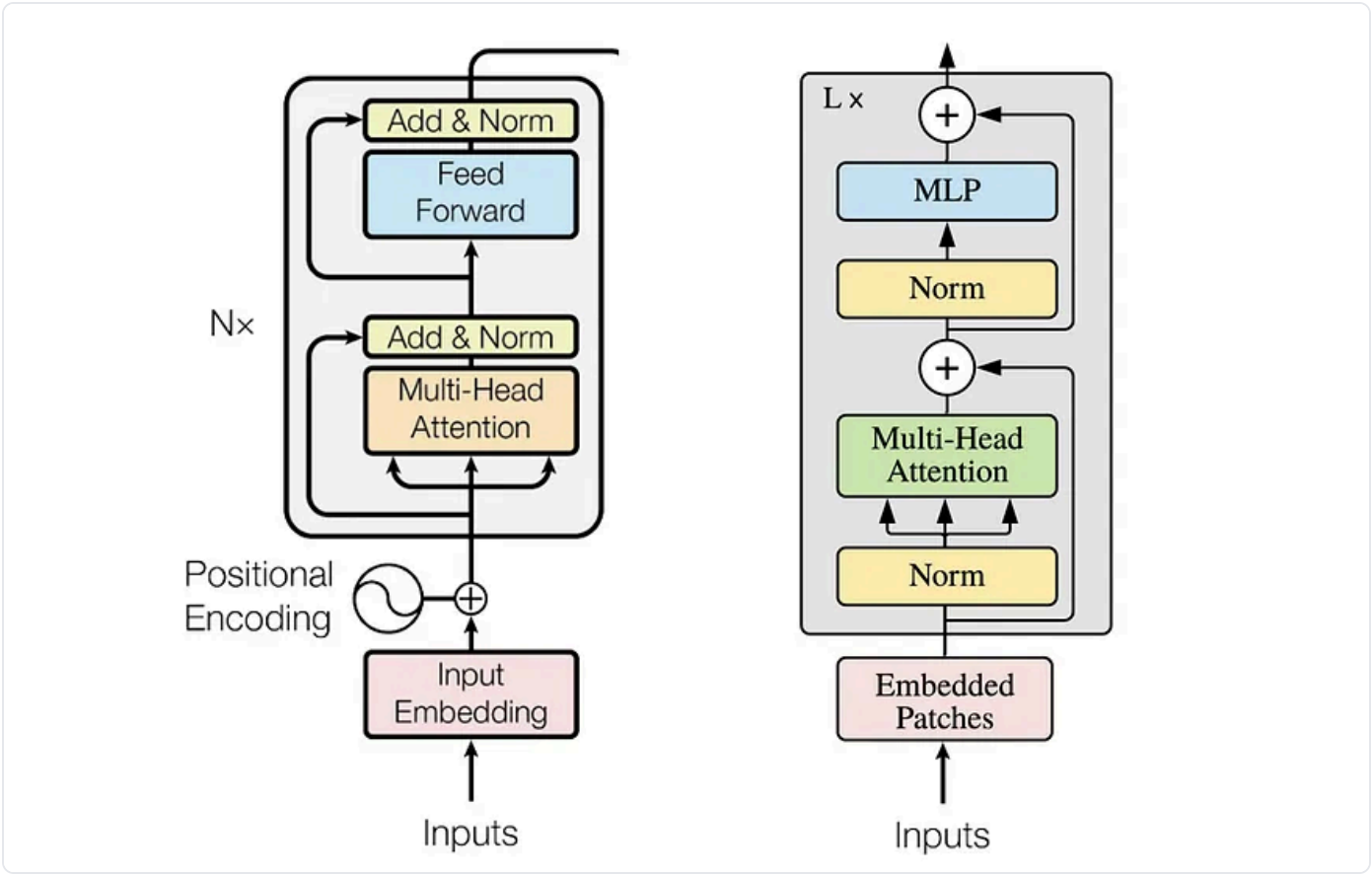
1.1.4、ViT 的核心优势

ViT 相较于传统 CNN 具备以下显著优势：

1. **全局建模能力强**：自注意力机制可直接建模任意空间位置之间的关系。
2. **结构更简洁**：无需复杂的卷积堆叠结构，架构统一、模块化程度高。
3. **可扩展性优异**：模型性能随着参数规模和数据规模持续提升。

1.1.5、Transformer Encoder 和 ViT 的区别

下图比较了原始论文《Attention is All You Need》中的 Transformer encoder 层与 ViT 模型，本质上是 **Post-LayerNorm 设计与 Pre-LayerNorm** 设计之间的区别。



Transformer Encoder 和 Vit 网络结构区别，**Post-LN（左图）**：先做子层再归一化，符合最初设计，但深层训练不稳定。**Pre-LN（右图）**：先归一化再做子层，残差路径更“纯”，梯度传播更顺畅，更适合深层模型。

1. 标准 Transformer（Post-LayerNorm）

原始 Transformer（Vaswani et al.， 2017）使用的是 **Post-LN** 结构，公式表示为：

$$\mathbf{x}_{l+1} = \mathrm{LayerNorm}(\mathbf{x}_l + \mathrm{Sublayer}(\mathbf{x}_l))$$

比如在自注意力层中：

$$\mathbf{x}_{l+1} = \mathrm{LayerNorm}(\mathbf{x}_l + \mathrm{MultiHeadAttn}(\mathbf{x}_l))$$

这就是 **Post-LN**，LayerNorm 在残差相加之后。

2.Pre-LayerNorm：把 LN 放到前面

后来很多工作（如 GPT、BERT 的改进版本、Deep Transformer）发现训练深层模型时 Post-LN 不稳定，于是公式为：

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathrm{Sublayer}(\mathrm{LayerNorm}(\mathbf{x}_l))$$

也就是：

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathrm{MultiHeadAttn}(\mathrm{LayerNorm}(\mathbf{x}_l))$$

这就是 **Pre-LN。**

3.区别（从梯度传播看）

- Post-LN 的问题

在 Post-LN 架构中，LayerNorm 位于残差连接之后：

$$\mathbf{x}_{l+1} = \mathrm{LN}\big(\mathbf{x}_l + f(\mathbf{x}_l)\big)$$

反向传播时，梯度必须通过 LayerNorm 才能传回到下层：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x} l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{l+1}} \cdot \frac{\partial \mathrm{LN}}{\partial(\cdot)} \cdot \left(I + \frac{\partial f}{\partial \mathbf{x}_l}\right)$$

由于 LayerNorm 会对特征进行归一化并引入缩放与偏移，其可能改变梯度的尺度和方向。随着网络加深（如超过 24 层），这种尺度变换在多层堆叠后容易导致梯度消失或爆炸，从而造成训练不稳定。

- Pre-LN 的优势

在 Pre-LN 架构中，LayerNorm 位于子层之前：

$$\mathbf{x}_{l+1} = \mathbf{x}_l + f(\mathrm{LN}(\mathbf{x}_l))$$

此时，残差路径是一条**纯恒等映射。**反向传播时，梯度可以直接沿残差分支无衰减地传播：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x} l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{l+1}} + \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{l+1}} \cdot \frac{\partial f(\mathrm{LN}(\mathbf{x}_l))}{\partial \mathbf{x}_l}$$

其中第一项对应残差连接带来的“直通梯度”，保证了即使子层梯度变小，主梯度路径仍然稳定存在。这使得深层 Transformer（如 48 层、96 层）在不使用复杂初始化或学习率调度的情况下，也能更稳定地训练。

1.1.6、ViT 代码实现

1. image2token

```
from PIL import Image
import torch
import torch.nn as nn
import torchvision.transforms as transforms

# ----------------------
# 1. 读取并预处理图像
# ----------------------
image = Image.open('input_demo.jpg').convert('RGB')
resized_image = image.resize((224, 224))

transform = transforms.Compose([
    transforms.ToTensor(),  # (C, H, W)
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

x = transform(resized_image)          # (3, 224, 224)
x = x.unsqueeze(0)                    # -> (1, 3, 224, 224)

print("输入图像张量形状:", x.shape)

# ----------------------
# 2. 切成 patch 并展平
# ----------------------
patch_height = 16
```

```
patch_width = 16

# (1, 3, 224, 224)
b, c, H, W = x.shape
# (16, 16)
p1, p2 = patch_height, patch_width
# 14
h = H // p1
w = W // p2

# 1) 拆成 patch 网格（1, 3, 14, 16, 14, 16)
x = x.view(b, c, h, p1, w, p2)

# 2) 调整维度顺序 -> (b, h, w, p1, p2, c) (1, 14, 14, 16, 16, 3)
x = x.permute(0, 2, 4, 3, 5, 1)

# 3) 拉平成 patch 向量 -> (b, num_patches, patch_dim)
x = x.reshape(b, h * w, p1 * p2 * c)

print("Patch token 序列形状（映射前）:", x.shape)

# ----------------------
# 3. 添加 FC 层，映射到统一 token 维度
# ----------------------
patch_dim = p1 * p2 * c      # 16*16*3 = 768
embed_dim = 768              # Transformer 中 token 的维度（可以自己设，比如 512、768、1024）

fc = nn.Linear(patch_dim, embed_dim)

# 对每个 patch 向量做线性映射
x = fc(x)   # (b, num_patches, embed_dim)

print("Patch token 序列形状（映射后）:", x.shape)
```

2. ViT encoder

```
import torch
from torch import nn
```

```
from math import sqrt as msqrt

# GELU激活函数
def gelu(x):
    return 0.5 * x * (1. + torch.erf(x / msqrt(2.)))

# =======================
# 缩放点积注意力机制
# =======================
class ScaledDotProductAttention(nn.Module):
    def __init__(self, d_k):
        super(ScaledDotProductAttention, self).__init__()
        self.d_k = d_k

    def forward(self, Q, K, V):
        '''
        Q: [batch, n_heads, seq_len, d_k]
        K: [batch, n_heads, seq_len, d_k]
        V: [batch, n_heads, seq_len, d_v]
        '''
        scores = torch.matmul(Q, K.transpose(-1, -2)) / msqrt(self.d_k)
        attn = nn.Softmax(dim=-1)(scores)
        context = torch.matmul(attn, V)
        return context

# =======================
# 多头注意力机制
# =======================
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model=768, n_heads=12, d_k=64, d_v=64):
        super(MultiHeadAttention, self).__init__()
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_k
        self.d_v = d_v

        self.W_Q = nn.Linear(d_model, d_k * n_heads, bias=False)
        self.W_K = nn.Linear(d_model, d_k * n_heads, bias=False)
        self.W_V = nn.Linear(d_model, d_v * n_heads, bias=False)
        self.fc = nn.Linear(n_heads * d_v, d_model, bias=False)
```

```
        self.attn = ScaledDotProductAttention(d_k)

    def forward(self, Q, K, V):
        '''
        Q, K, V: [batch, seq_len, d_model]
        '''
        batch = Q.size(0)

        per_Q = self.W_Q(Q).view(batch, -1, self.n_heads, self.d_k).transpose(1, 2)
        per_K = self.W_K(K).view(batch, -1, self.n_heads, self.d_k).transpose(1, 2)
        per_V = self.W_V(V).view(batch, -1, self.n_heads, self.d_v).transpose(1, 2)

        context = self.attn(per_Q, per_K, per_V)
        context = context.transpose(1, 2).contiguous().view(batch, -1, self.n_heads * self.d_v)

        output = self.fc(context)
        return output

# =======================
# 前馈神经网络
# =======================
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model=768, d_ff=3072):
        super(FeedForwardNetwork, self).__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
        self.gelu = gelu

    def forward(self, x):
        '''
        x: [batch, seq_len, d_model]
        '''
        x = self.fc1(x)
        x = self.gelu(x)
        x = self.fc2(x)
        return x

# =======================
# Encoder Layer（重点）
```

```
# ========================
class EncoderLayer(nn.Module):
    def __init__(self, d_model=768, n_heads=12, d_k=64, d_v=64, d_ff=3072):
        super(EncoderLayer, self).__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.enc_attn = MultiHeadAttention(d_model, n_heads, d_k, d_v)
        self.ffn = FeedForwardNetwork(d_model, d_ff)

    def forward(self, x):
        '''
        x: [batch, seq_len, d_model]
        '''
        residual = x
        x = self.norm1(x)
        x = self.enc_attn(x, x, x) + residual

        residual = x
        x = self.norm2(x)
        x = self.ffn(x) + residual
        return x

# ========================
# ViT 主体结构
# ========================
class ViT(nn.Module):
    def __init__(self, embed_dim=768, num_patches=196, num_classes=1000,
                 depth=12, num_heads=12, mlp_ratio=4.0):
        super().__init__()

        self.embed_dim = embed_dim

        # [CLS] token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))

        # 位置编码
        self.pos_embed = nn.Embedding(num_patches + 1, embed_dim)

        # Encoder layers
        d_ff = int(embed_dim * mlp_ratio)
```

```
        self.layers = nn.ModuleList([
            EncoderLayer(
                d_model=embed_dim,
                n_heads=num_heads,
                d_k=embed_dim // num_heads,
                d_v=embed_dim // num_heads,
                d_ff=d_ff
            ) for _ in range(depth)
        ])

        # 分类头
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        """
        x: (B, num_patches, embed_dim)
        """
        B, N, D = x.shape

        # 拼接 cls token
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1)  # (B, N+1, D)

        # 加位置编码
        pos_ids = torch.arange(N + 1, device=x.device).unsqueeze(0)  # (1, N+1)
        x = x + self.pos_embed(pos_ids)

        # Transformer Encoder
        for layer in self.layers:
            x = layer(x)

        # 分类
        x = self.norm(x)
        cls_out = x[:, 0]
        logits = self.head(cls_out)
        return logits

# ======================
# 测试
```

```
# ========================
if __name__ == "__main__":
    # 测试 ViT
    num_patches = 196
    x = torch.randn(1, num_patches, 768)
    model = ViT(embed_dim=768, num_patches=num_patches, num_classes=1000)
    logits = model(x)
    print("ViT logits shape:", logits.shape)
```

1.2、MLLM（以 Qwen3-vl 为例）

> qwen3-vl Technical Report: https://arxiv.org/pdf/2511.21631

1.2.1、MLLM 主流模型结构

> https://arxiv.org/pdf/2304.08485

下图展示了一个典型的多模态语言模型结构，用于将图像与语言指令融合并生成自然语言回答。整体流程可以理解为：先把图像转换为模型能理解的表示，再与文本一起送入语言模型，最终生成语言输出。

首先，输入图像 （X_v） 经过视觉编码器（如 ViT）提取为高维视觉特征 （ Z_v）。这些特征仍处在视觉模型的表示空间中，不能直接被语言模型使用，因此通过一个线性投影层 （W），将其映射到与语言模型嵌入维度一致的空间，得到视觉表示 （ H_v）。这一步的作用是让“视觉信息说语言模型能听懂的话”。

与此同时，语言指令 （ X_q） 被语言模型编码（tokenizer + embedding）为对应的隐藏表示 （ H_q），表示问题或指令的语义内容。此时，模型已经分别获得了来自图像和文本的两种语义表示。

接下来，模型将视觉表示 （ H_v） 与语言表示 （ H_q） 拼接在一起，作为统一的输入送入语言模型 （ f_ϕ）。语言模型在同一序列中同时感知图像语义和语言语义，并基于这种联合信息进行推理和生成，最终输出语言响应 （ X_a）。

从整体上看，这一结构的核心思想是：通过投影层对齐视觉特征与语言特征，使语言模型能够像处理文本一样处理图像信息，从而实现“看图 + 听话 + 作答”的统一建模。

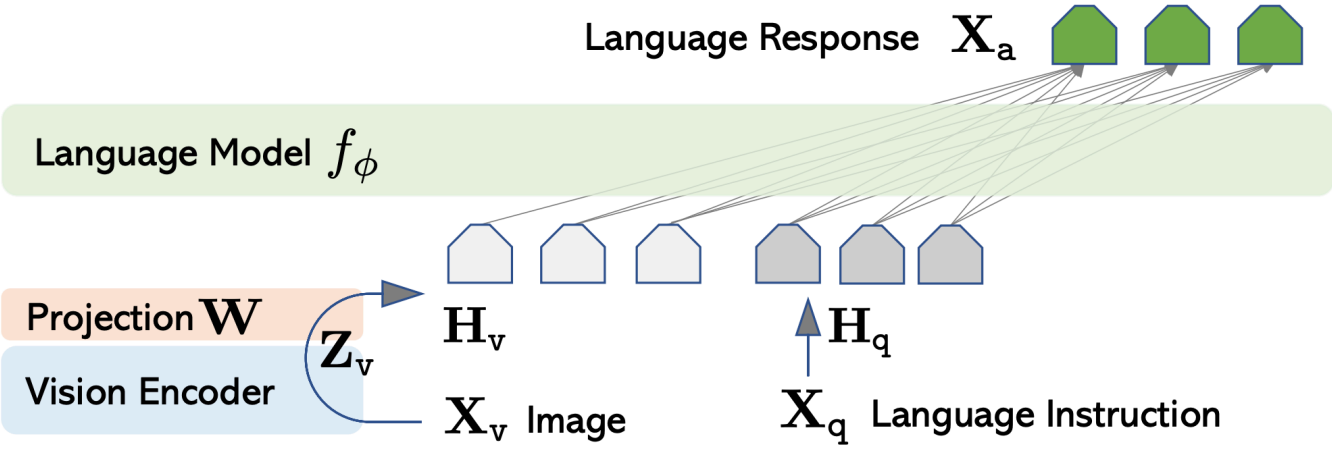
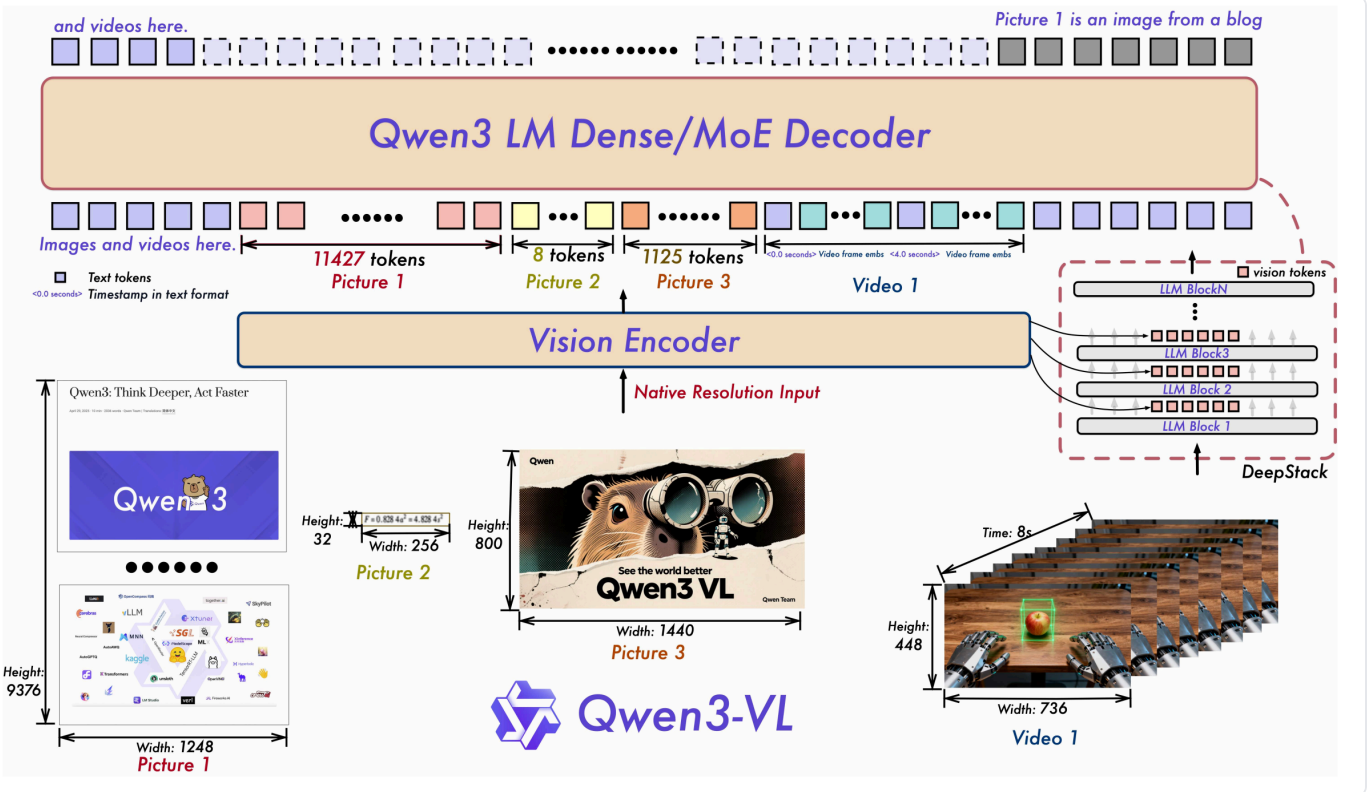


Figure 1: LLaVA network architecture.

1.2.2、Qwen3-VL 模型结构

Qwen3-VL 沿用“视觉编码器→视觉-语言投影层 →LLM 解码器”的经典 VLM 范式，由三个模块组成：1、视觉编码器；2、视觉-语言投影层 ； 3、语言模型解码器。具体如下图所示：



1. 视觉编码器：从像素到视觉特征表示

- 功能

- 视觉编码器负责将输入的图像或视频帧转化为高维向量序列，使其能够被后续语言模型处理。

- 架构与参数

- 基于 SigLIP-2 的 Vision Transformer 架构；

---

2. 视觉-语言投影层： 将视觉特征映射为语言可处理形式

- 功能

- 投影层将视觉编码器输出的视觉特征映射到与语言模型隐藏空间一致的表示，使视觉信息可以作为语言 token 输入到 LLM 中。

- 架构与参数

- 使用两层 MLP 将视觉特征映射至 LLM 隐藏维度（如 4096 维）。

---

3. LLM 解码器：统一理解、推理与生成模块

- 功能

- LLM 解码器接收文本 token 与视觉 token 的混合序列，完成理解、推理与自然语言生成任务。

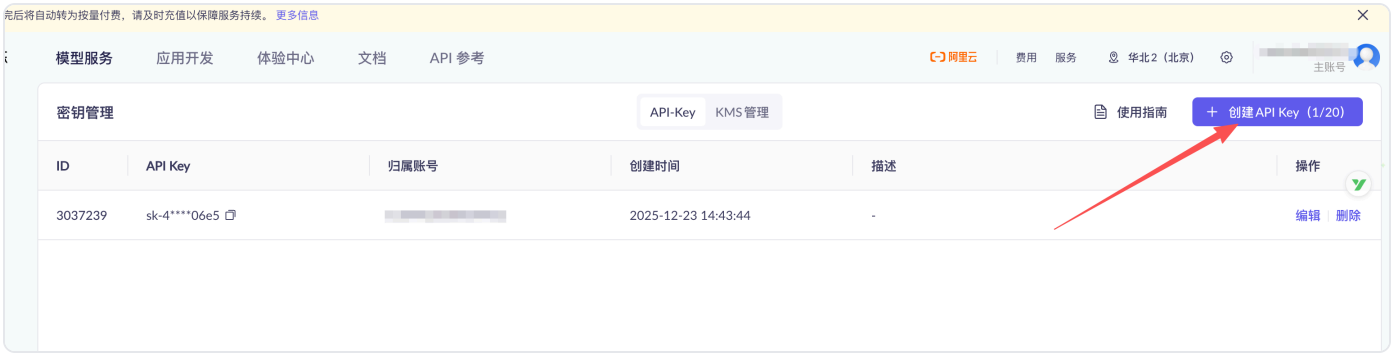
- 架构与参数

- 基于 Qwen3 Transformer 解码器结构； - 支持稠密与 MoE 两种变体；

1.2.3、调用阿里 api 尝试

- 注册你的 api key 步骤

- 使用支付宝扫码注册：api-key 获取网站 - 创建 api key



- 复制生成的 api key 到这里即可 `api_key="替换为你的api key"`

```
from dashscope import MultiModalConversation
# import dashscope
# pip install dashscope
# 若使用新加坡地域的模型，请取消下列注释
# dashscope.base_http_api_url = "https://dashscope-intl.aliyuncs.com/api/v1"

image_path = "tony_dataset/img/test/1989609.jpg"
messages = [
                {'role':'user',
                'content': [{'image': image_path},
                            {'text': '图中描绘的是什么景象?'}]}]
response = MultiModalConversation.call(
    # 若没有配置环境变量，请用百炼API Key将下行替换为：api_key="sk-xxx"
    # 新加坡和北京地域的API Key不同。获取API Key：https://help.aliyun.com/zh/model-studio/get-a
    api_key="替换为你的api key",
    model='qwen3-vl-plus',  # 此处以qwen3-vl-plus为例，可按需更换模型名称。模型列表：https://hel
    messages=messages)
print(response.output.choices[0].message.content[0]["text"])
```

二、项目实践

> **讲解视频：链接 1，对应到 2.6.4；**

> 代码下载地址：百度网盘链接

2.1、比赛介绍

城市管道系统是城市基础设施的重要组成部分，承担着输送与排放等基本功能。由于管道大多位于地下，其运行状态难以直接观察，一旦发生缺陷，往往难以及时发现。因此，利用技术手段对管道进行自动检测，对于保障城市基础设施的正常运行具有重要意义。

该项目来源于 ECCV（European Conference on Computer Vision）竞赛体系中的 UrbanPipe 挑战赛。ECCV 是计算机视觉领域的重要国际会议之一。UrbanPipe 挑战赛基于真实城市管道数据，要求参赛者构建模型，从管道检测视频中识别缺陷类型，实现细粒度、多标签的异常识别。

2.2、任务定义

本任务属于多标签分类问题。与传统的单标签分类不同，多标签分类允许一个样本同时对应多个类别。由于管道环境复杂，一个视频中可能同时存在多种缺陷。例如，同一段视频中既可能出现结构变形，也可能同时存在裂缝或沉积。因此，每个视频对应的是一个标签集合，而不是单一类别。

本任务的目标是：给定一个管道检测视频，预测其中存在的所有缺陷类别。模型需要对每一种缺陷分别进行判断，并输出对应的预测结果。

2.3、数据集

> 0、在代码中，我会给一小部分数据，用于调试和学习使用，完整的数据集下载方式：

> 1、如果需要更多数据集，从百度网盘下载，一个文件夹下是 500 个视频：下载链接

> 2、完整数据集：从 OpenDataLab 网站下载：track1-Video_Defect_Classification

城市管道 UrbanPipe 数据集（如图 1 所示）：一种细粒度且多标签的视频数据集，从现实世界的城市管道系统中的各种快速查看检测设备中收集而来的。

此处为动图（内容见飞书原文）

图 2-1 数据缩写字母表示：ZC 正常 CJ 沉积 CK 错口 ZW 障碍物 JG 结垢 BX 变形 PL 破裂 CR 穿入 FS 腐蚀 SG 树根 FZ 浮渣 TJ 脱节 AJ 暗接 CQ 残墙 TL 脱落 SL 渗漏 QF 起伏

- **数据收集与标注**

UrbanPipe 数据集包含 1 个正常类别和 16 个缺陷类别，共计 9600 个视频，所有视频的总时长超过 55 小时，视频的持续时间范围从 0.7 秒到 177.4 秒不等，每个视频的持续时间为 20.7 秒。由于管道情况复杂，多个缺陷往往同时出现，因此每个视频被 1 到 5 个类别进行了标注，平均为 1.4 个，这些视频均采集自真实城市管道，并由专业工程师进行标注。数据的统计信息如图 2-2 所示。



Dataset examples

Classes	1 normal class 16 defect classes
Classification Type	Multi-Label
Video numbers	9609
Instance Labels Num Range	1 – 5 labels
Average Labels Num	1.4 labels
Total Duration	55H
Average Duration	20.7s
Duration Range	0.7s – 385.2s

Dataset statistics

图 2-2 数据集统计信息

整体的数据呈现出自然的长尾分布，其中，0 正常、1 沉积、2 腐蚀、3 障碍物、4 结垢、5 树根、6 破裂、7 变形、8 异物穿入、9 浮渣、10 脱节、11 暗接、12 残墙、13 渗漏、14 起伏、15 错口、16 脱落 ，具体每个类别的分布情况如图 2-3 所示。

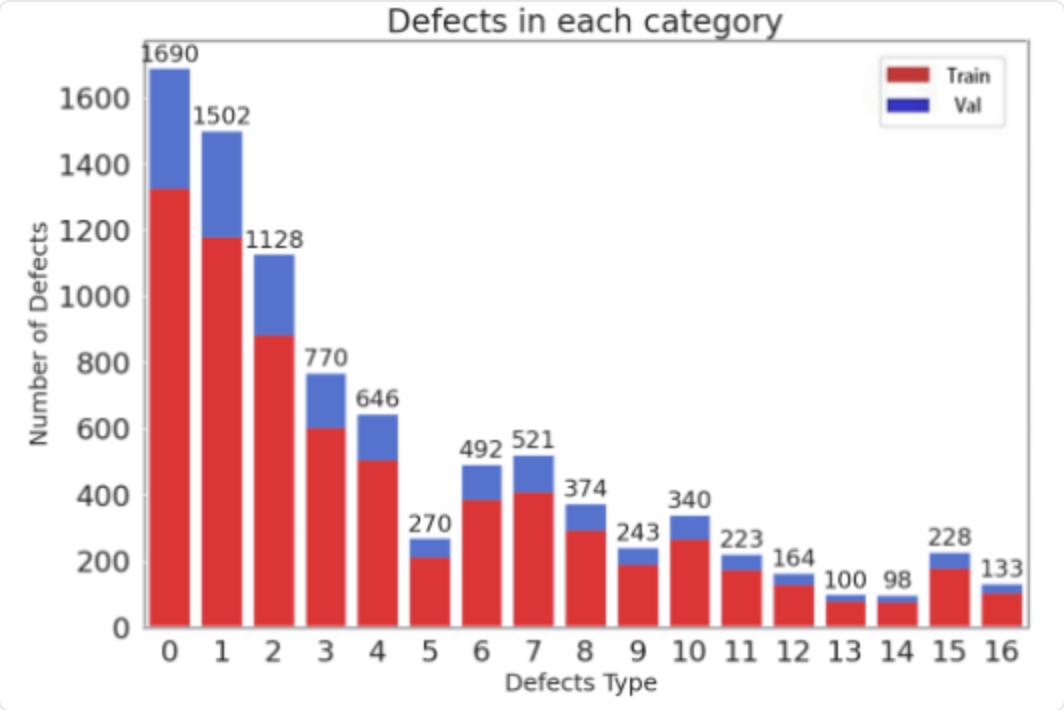


图 2-3 数据集的类别分布情况

2.4、评价指标

> Ref: https://jonathan-hui.medium.com/map-mean-average-precision-for-object-detection-45c121a31173

在多标签分类任务中，一个样本可能同时属于多个类别。模型会为每个类别输出一个置信度分数，表示该样本属于该类别的可能性。如果仅使用准确率作为评价指标，往往无法完整反映模型性能。例如，一张图片的真实标签为“猫、狗”，而模型只预测出“猫”，虽然预测部分正确，但模型仍然漏检了“狗”。因此，需要一种能够同时反映预测准确性和检出完整性的评价指标，mAP（mean Average Precision）便是常用的标准。

mAP 的计算基于 Precision（精确率）和 Recall（召回率）。精确率表示在所有预测为正的样本中，有多少是真正的正样本；召回率表示在所有真实正样本中，有多少被模型成功识别。通过改变预测阈值，可以得到多组 Precision 和 Recall 的对应关系，并在坐标系中绘制为 PR 曲线。PR 曲线反映了模型在不同判断标准下的性能变化。

为了说明 AP 的计算过程，可以考虑单个类别“猫”。假设数据集中共有 10 张图片，其中 5 张真实包含“猫”。模型为每张图片输出一个预测分数，并将所有图片按分数从高到低排序（如图 2-4

所示）。随后，从第一张开始逐步向下计算。在每一个位置，统计当前为止识别出的正样本数量，并计算对应的 Precision 和 Recall（如图 2-4 所示）。例如，当遍历到第 3 张图片时，如果其中有 2 张是真实正样本，则 Precision 为 2/3，Recall 为 2/5。以 Recall 为横轴、Precision 为纵轴，可以得到 PR 曲线上的一个点。重复这一过程，即可得到完整的 PR 曲线（如图 2-5 所示）。

Rank	Correct?	Precision	Recall
1	True	1.0	0.2
2	True	1.0	0.4
3	False	0.67	0.4
4	False	0.5	0.4
5	False	0.4	0.4
6	True	0.5	0.6
7	True	0.57	0.8
8	False	0.5	0.8
9	False	0.44	0.8
10	True	0.5	1.0

图 2-4 按照模型对每张图中“猫”类别预测的概率从高到低排序，以此计算 P 和 R

平均精度（AP）定义为 PR 曲线下的面积。该面积反映了模型在不同召回水平下维持高精确率的能力。AP 的取值范围在 0 到 1 之间，数值越大，说明模型在该类别上的性能越好。

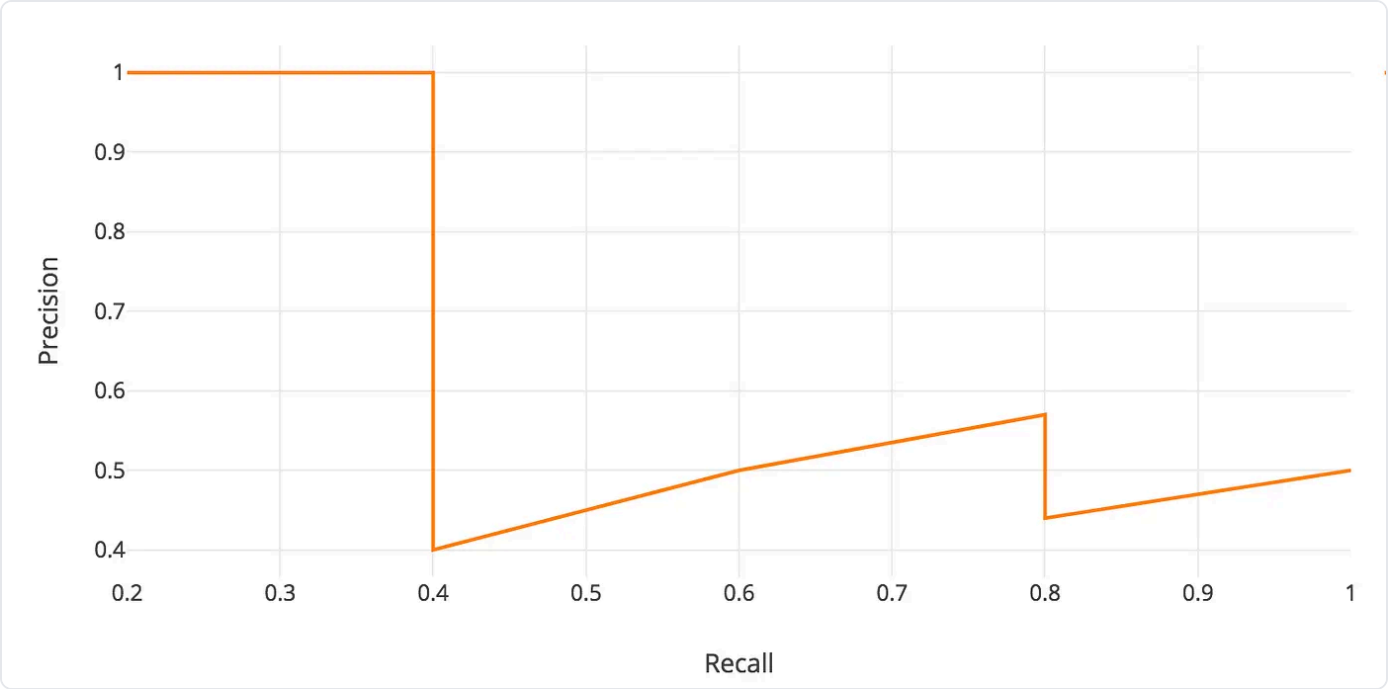


图 2-5 PR 曲线，根据 PR 点绘制而成， PR 曲线下的面积定义为平均精度（AP）

在计算出每个类别的 AP 后，对所有类别的 AP 取平均，即得到 mAP。例如，如果三个类别的 AP 分别为 1.00、0.83 和 0.90，则：mAP = （1.00 + 0.83 + 0.90） / 3 = 0.91。

2.5、任务难点

多标签。与传统单标签分类不同，模型需要同时判断多个缺陷是否存在，而不是仅输出一个类别。这要求模型能够独立学习各类别的判别特征，并在预测时对每个类别分别做出可靠判断，避免类别之间的相互干扰。

长尾分布。部分缺陷类别样本数量较少，使得模型难以充分学习其特征表示。在训练过程中，模型更容易偏向于样本数量较多的常见类别，而对少数类别的识别能力不足，从而影响整体性能的均衡性。

细粒度特征。不同缺陷所依赖的信息维度存在差异。部分缺陷主要体现在空间结构上，例如形状变化或结构异常；而另一些缺陷则更多表现为随时间变化的过程特征。这要求模型不仅能够理解单帧图像的空间信息，还需要建模时间维度上的动态变化。

2.6、实现方法

图 2-6 展示了针对城市管道视频异常检测任务的四种不同建模方法，分别从图像理解、视频理解、超图理解以及多模态大模型四个角度来进行建模。

第一种方法是基于图片理解任务的方法。在这种方法中，视频首先被分解为一系列连续的图片帧，每一帧被视为一张独立的图像，并输入图像分类网络进行预测。模型分别对每一帧输出各类别的预测结果，最后将所有帧的预测结果进行融合（如平均），得到整个视频的分类结果。这种方法实现简单，训练成本较低，并且可以直接利用成熟的图像分类模型。然而，该方法忽略了帧与帧之间的时间关系，因此对于依赖时间变化的异常，其识别能力有限。

第二种方法是基于视频理解任务的方法。在这种方法中，模型直接以视频片段作为输入，而不是单独的图片帧。视频分类网络能够同时学习空间信息和时间信息，从而更好地理解视频中的动态变化。例如，一些缺陷可能表现为随时间逐渐变化的结构异常，这类特征只有在连续帧中才能被有效捕捉。相比逐帧方法，视频理解方法能够更全面地建模视频内容，但其计算成本更高，对训练数据的需求也更大。

第三种方法是基于超图理解的方法。该方法通过从视频中选取多个关键帧（比如 9 张），并将这些帧拼接成一张新的图像，称为“超图”，然后使用图像分类网络进行预测。这样可以在保持图像模型高效性的同时，将多个时间点的信息整合到同一个输入中，使模型能够同时观察多个关键时刻的空间结构。这种方法在计算效率和时间信息利用之间取得了良好的平衡，特别适用于那些主要依赖空间结构特征的缺陷识别任务。

第四种方法是基于多模态大模型的方法。多模态大模型能够同时处理视觉信息和语言信息，具备更强的语义理解能力。例如，可以将视频帧输入多模态模型，并结合文本提示（如缺陷类别描述），让模型根据视觉内容判断视频中是否存在对应缺陷。这类模型通常基于大规模预训练，具备较强的泛化能力和零样本学习能力，能够在数据较少或类别不均衡的情况下仍保持较好的性能。



图 2-6 实现方法一览图

2.6.1、多标签分类任务的损失函数

在多标签分类任务中，一个样本可能同时属于多个类别。例如，一段管道视频中可能同时存在“裂缝”和“腐蚀”两种缺陷。这与传统分类任务不同。在传统的单标签分类中，每个样本只能属于一个类别，各类别之间是互斥的；而在多标签分类中，各类别之间相互独立，可以同时存在。因此，损失函数的选择需要适应这一特性。

在单标签分类任务中，通常使用交叉熵损失函数。该损失函数基于 softmax 函数，将模型输出的分数转换为概率分布。softmax 的特点是所有类别的概率之和等于 1，这意味着模型必须在所有类别中选择一个最可能的类别。

在多标签分类问题中，单个样本可能同时属于多个类别，每个类别的存在与否相互独立，因此对类别概率进行归一化的约束显然不适用。为此，BCEWithLogitsLoss 应运而生，它将 sigmoid 激活函数与二元交叉熵损失相结合，对每个类别独立建模。设单个样本的真实标签为 （$\mathbf{y} = [y_1, y_2, \ldots, y_C]$），其中 （$y_c \in 0, 1$） 表示类别 （c） 是否存在，模型输出的 logits 为 （$\mathbf{z} = [z_1, z_2, \ldots, z_C]$），则 BCEWithLogitsLoss 对每个类别的损失定义为

$$\ell_c = -\big(y_c \log \sigma(z_c) + (1 - y_c) \log(1 - \sigma(z_c))\big),$$

其中 （$\sigma(z) = \frac{1}{1+e^{-z}}$） 为 sigmoid 函数，将 logits 映射为概率。整个样本的损失可取各类别损失的平均或总和，即

$$\mathcal{L} = \frac{1}{C} \sum_{c=1}^{C} \ell_c = -\frac{1}{C} \sum_{c=1}^{C} \big(y_c \log \sigma(z_c) + (1 - y_c) \log(1 - \sigma(z_c))\big).$$

从本质上讲，BCEWithLogitsLoss 将多标签问题视作对每个类别独立的二分类问题，通过对每个类别的存在与否进行概率建模，既符合多标签任务的语义独立性，也保证了损失函数在数学上的可微性和优化的稳定性。

- 代码实现

```
import torch
import torch.nn as nn
# 1. 创建输入数据
# 两个样本，每个样本3维特征
input = torch.tensor([
                [0.1, 0.2, 0.3, 1.4],
                [0.4, 0.5, 0.6, 0.2]
```

```
            ], dtype=torch.float32)

# 每个样本对应3个标签（多标签）
target = torch.tensor([
                [1, 0, 1],
                [0, 1, 1]
            ], dtype=torch.float32)

# 2. 创建模型
model = nn.Linear(in_features=4, out_features=3)

# 3. 前向传播
model_out = model(input)

print("模型输出 logits: ")
print(model_out)

# 4. 定义损失函数
criterion = nn.BCEWithLogitsLoss()

# 5. torch库函数计算损失
official = criterion(model_out, target)

def my_bce_with_logits(logits, targets):
    prob = torch.sigmoid(logits)
    loss = -(
        targets * torch.log(prob + 1e-8)
        +
        (1-targets) * torch.log(1-prob + 1e-8)
    )
    return loss.mean()

# 6. 自己定义的进行计算
custom = my_bce_with_logits(model_out, target)

print(official)
print(custom)
```

2.6.2、基于视频帧的方法

如图 2-7 所示，基于图像帧的方法将动作识别问题转化为图像分类问题，从而避免直接对复杂的时序信号进行建模。在这一框架中，一个视频不再被视为连续变化的整体，而是被看作由若干离散图像帧组成的集合。具体而言，首先对视频进行均匀采样，**在完整视频序列中等间隔抽取固定数量的帧图像（本实验中统一为 16 帧）**，以覆盖视频的主要内容。随后，将视频原有的类别标签赋予这 16 帧图像，使其共享同一语义标注，从而将视频分类任务转化为标准的图像分类任务。需要强调的是，无论在训练阶段还是推理阶段，模型处理的都不是视频中的全部帧，而是等间隔采样得到的这 16 帧。

在训练阶段，采样得到的 16 帧图像首先经过数据增强处理，例如随机裁剪、翻转与归一化等操作，以提升模型对尺度变化和外观变化的鲁棒性。增强后的图像被输入到图像分类网络中，如 ResNet 或 Vision Transformer 等模型，通过端到端方式进行优化。模型通过学习这些采样帧与类别标签之间的映射关系，获得对视频动作语义的判别能力。由于每个视频仅使用固定数量的关键帧参与训练，该方法在保证语义覆盖的同时，也兼顾了计算效率与训练稳定性。

在测试阶段，同样对视频进行等间隔采样，提取 16 帧输入模型，分别得到对应的类别概率分布。随后，将这 16 帧的预测结果在时间维度上进行融合，通常采用对概率取平均的方式，将帧级预测整合为视频级预测结果。通过这种基于采样帧的概率融合策略，可以在控制计算成本的同时，获得稳定可靠的整体预测结果。

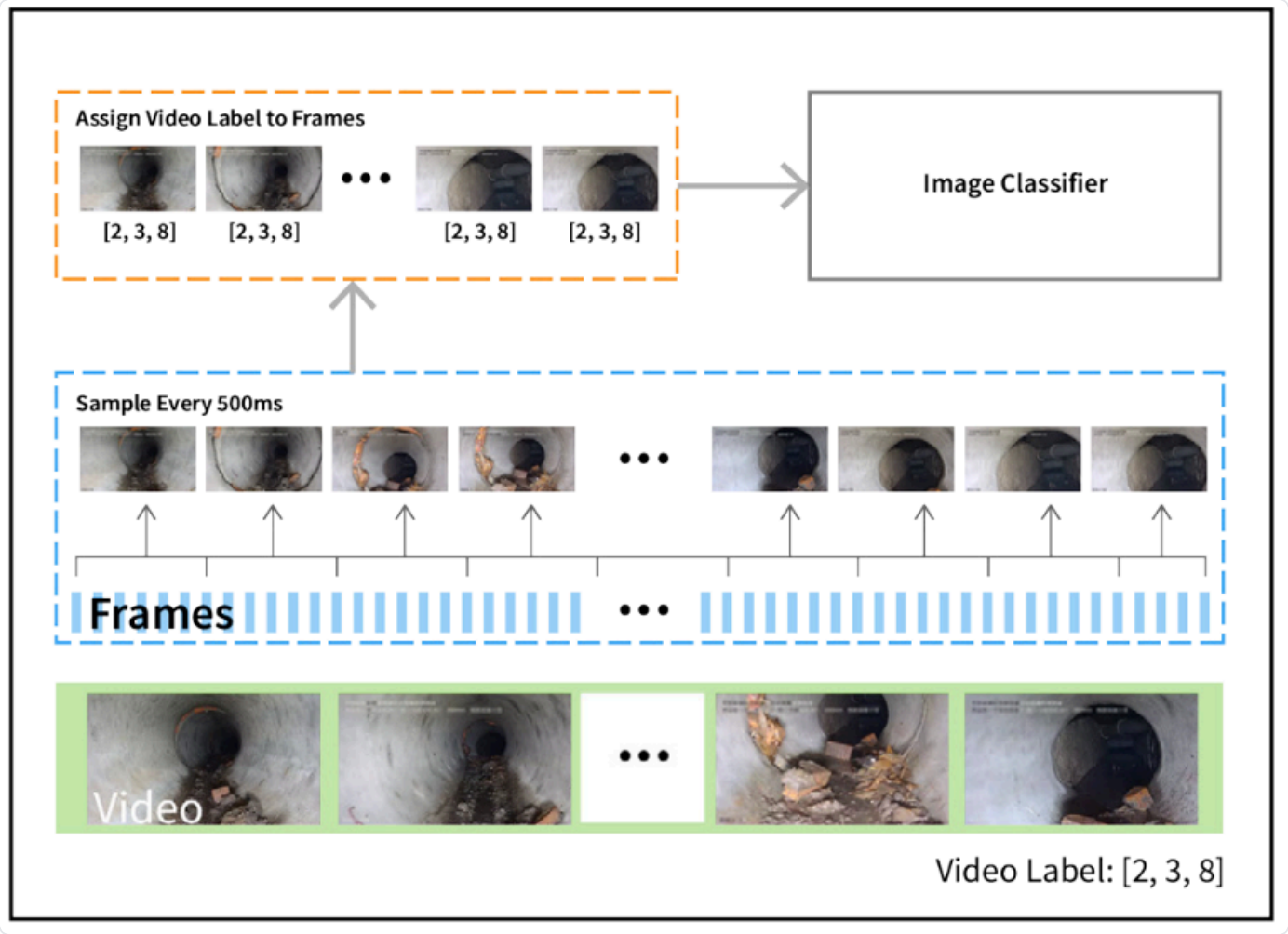


图 2-7 基于视频帧的方法结构示意图

(1) ViT

ViT 模型介绍见 1.1 部分，模型具体加载代码如下：

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.models as models

def modify_fc_layer(model, num_classes):
    """
    修改模型的最后分类层，使输出类别数量为 num_classes。
    """

    # Vision Transformer (ViT)
```

```
    if hasattr(model, 'heads') and hasattr(model.heads, 'head'):
        model.heads.head = nn.Linear(model.heads.head.in_features, num_classes)

    else:
        raise ValueError("The model does not contain a compatible final layer.")

    return model

def get_model(model_name="vit_b_16", num_classes=16, pretrained=False):

    print('==> model name {}'.format(model_name))
    # =========================
    # Vision Transformer (ViT)
    # =========================
    if model_name == "vit_b_16":
        weights = "IMAGENET1K_V1" if pretrained else None
        model = models.vit_b_16(weights=weights)
        model = modify_fc_layer(model, num_classes)

    elif model_name == "vit_b_32":
        weights = "IMAGENET1K_V1" if pretrained else None
        model = models.vit_b_32(weights=weights)
        model = modify_fc_layer(model, num_classes)

    elif model_name == "vit_l_16":
        weights = "IMAGENET1K_V1" if pretrained else None
        model = models.vit_l_16(weights=weights)
        model = modify_fc_layer(model, num_classes)

    else:
        raise ValueError("Unknown model value of : {}".format(model_name))

    return model

# =========================
# 测试代码
# =========================

if __name__ == "__main__":
```

```
    model = get_model(
        model_name="vit_b_16",
        num_classes=16,
        pretrained=True
    )

    print(model)

    # 测试 forward
    x = torch.randn(1, 3, 224, 224)
    y = model(x)

    print("output shape:", y.shape)
```

（2）代码

- 抽取视频帧

- 代码文件夹路径：urbanpipe_videoFrame - 等间隔抽取视频抽取 16 帧 - 代码位置：`urbanpipe_videoFrame/tools/video2images.py`

```
python3 ./tools/video2images.py \
--input_folder ../dataset/tony_data_video \
--output_folder ./dataset/tony_data_videoFrame
```

- **参数说明**

- 注意 `../dataset/` 和 `./dataset/` 不是一个位置 - `--input_folder`：视频文件夹路径 - `--output_folder`：抽取帧保存路径

- 训练

- 代码位置：urbanpipe_videoFrame/train.py

```
python3 ./train.py \
--base_path ./dataset/tony_data_videoFrame \
--label_file_train ../dataset/video_label_file/video_train.json \
--label_file_val ../dataset/video_label_file/video_val.json \
--model_name resnet18 \
```

```
--batch_size 24 \
--learning_rate 0.001 \
--epochs 20
```

- **参数说明**

- `--base_path`：视频帧数据目录 - `--label_file_train`：训练标签 JSON - `--label_file_val`：验证标签 JSON - `--model_name`：模型名称，支持：ResNet 系列：resnet18、resnet34、resnet50 等；ViT 系列：vit_b_16、vit_b_32、vit_l_16 - `--batch_size`: batch size - `--learning_rate`：学习率 - `--epochs`：训练轮数

2.6.3、基于视频的方法

如图 2-8 所示，基于视频的方法直接以连续帧片段作为输入，使网络能够同时建模空间特征与时间特征，从而更准确地理解视频中的动作语义。

具体而言，首先将原始视频解码为连续的图像帧序列，然后采用 均匀采样（Uniform Sampling）方法，从完整视频中等间隔提取固定长度的帧片段（如 8 帧），以保证采样片段能够覆盖视频的主要时序信息。随后，对采样得到的帧序列进行数据增强处理，并将其作为一个整体输入视频分类网络。

在训练阶段，采样得到的视频片段被输入网络，模型输出对应的类别概率分布，并使用多分类交叉熵损失函数进行端到端优化，使网络逐步学习视频片段与类别标签之间的映射关系，从而获得稳定的动作识别能力。在本实验中，我们采用 Video Vision Transformer（Video ViT）作为视频分类模型。该模型基于 Transformer 架构，通过自注意力机制对整个视频片段进行全局建模，使网络能够同时关注关键帧及其时间关系，从而有效提取动作相关的时空特征。

在推理阶段，采用与训练阶段一致的采样策略，从待测试视频中等间隔提取固定长度的帧片段（如 8 帧），并将其输入训练好的 Video Vision Transformer 模型中进行前向计算，得到该视频片段的类别概率分布。由于模型直接基于完整的视频片段进行时空联合建模，因此无需再进行帧级预测和额外的时间融合处理，网络输出结果即可作为视频级别的最终预测结果。

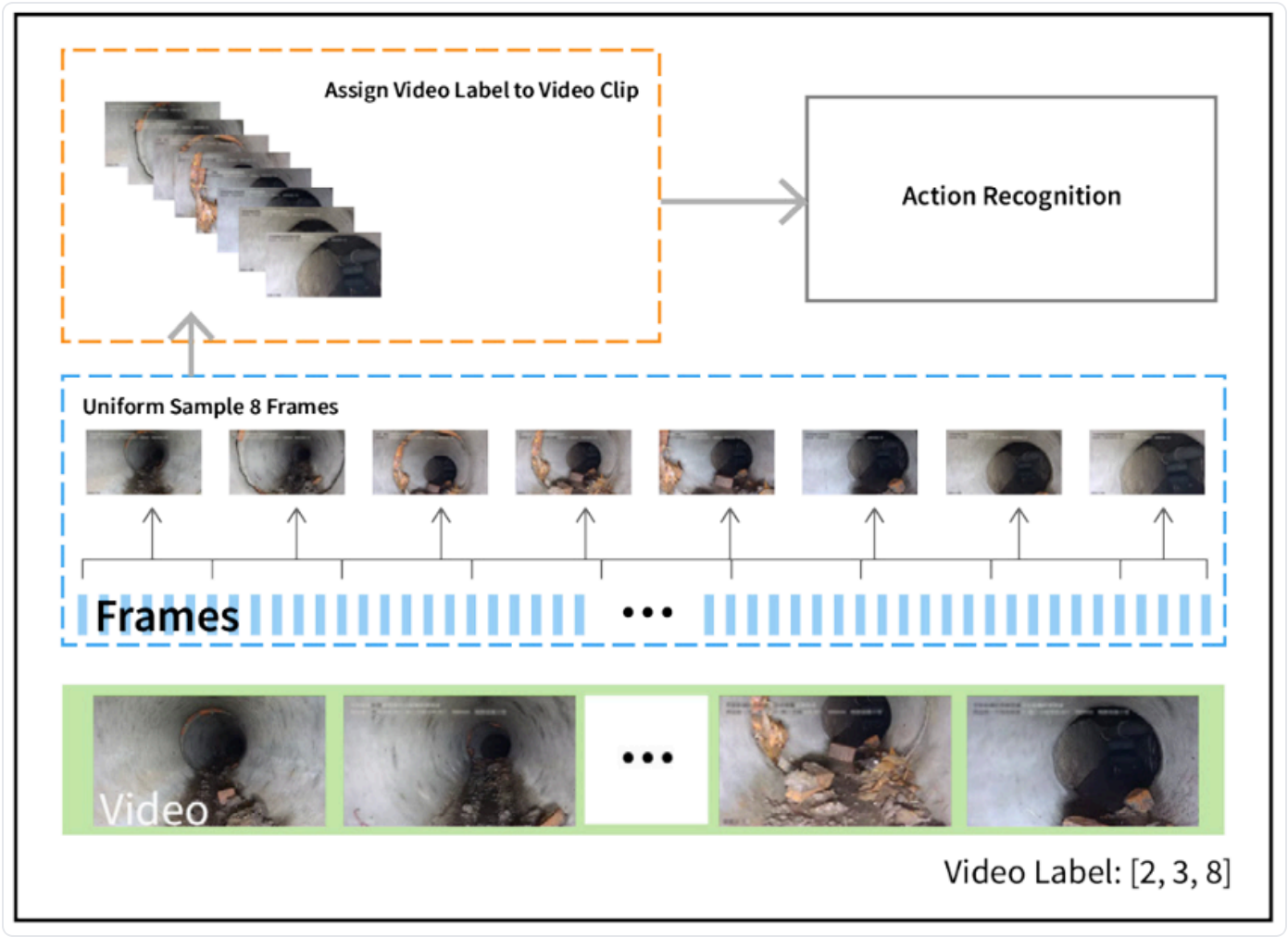


图 2-8 基于视频的方法。首先将视频转化为一帧帧图片，然后使用 Uniform 采样方法提取特定长度的片段并经过数据增强后，将其传递到具有提取时间特征和空间特征能力的视频分类网络中，最终使用多分类交叉熵损失函数进行端到端学习。

尽管该方法有效提取了时空特征，但该任务中，时序信息对本任务的贡献较小（例如，管道腐蚀的话，一张图片足以判断出来，而且还会比视频判断的更好，因为视频包含的噪音更多）。

(1) TimeSformer

> 原始 paper： https://arxiv.org/pdf/2102.05095

> 讲解视频：单独讲 TimeSformer 时间 0:49:18 开始： TimeSformer 讲解

> 代码实现参考：https://huggingface.co/docs/transformers/model_doc/timesformer

在视频理解任务中，TimeSformer 提出了一种基于自注意力机制的时空建模方法，其核心思想是将视频片段视为帧级图像块的序列，并在指定的时空邻域内对每个图像块执行自注意力计算。如图 2-9 展示了模型的整体结构图；图 2-10 中左图展示了 TimeSformer 模型的 bolck 块的组成，右图：时空自注意力方案的可视化。

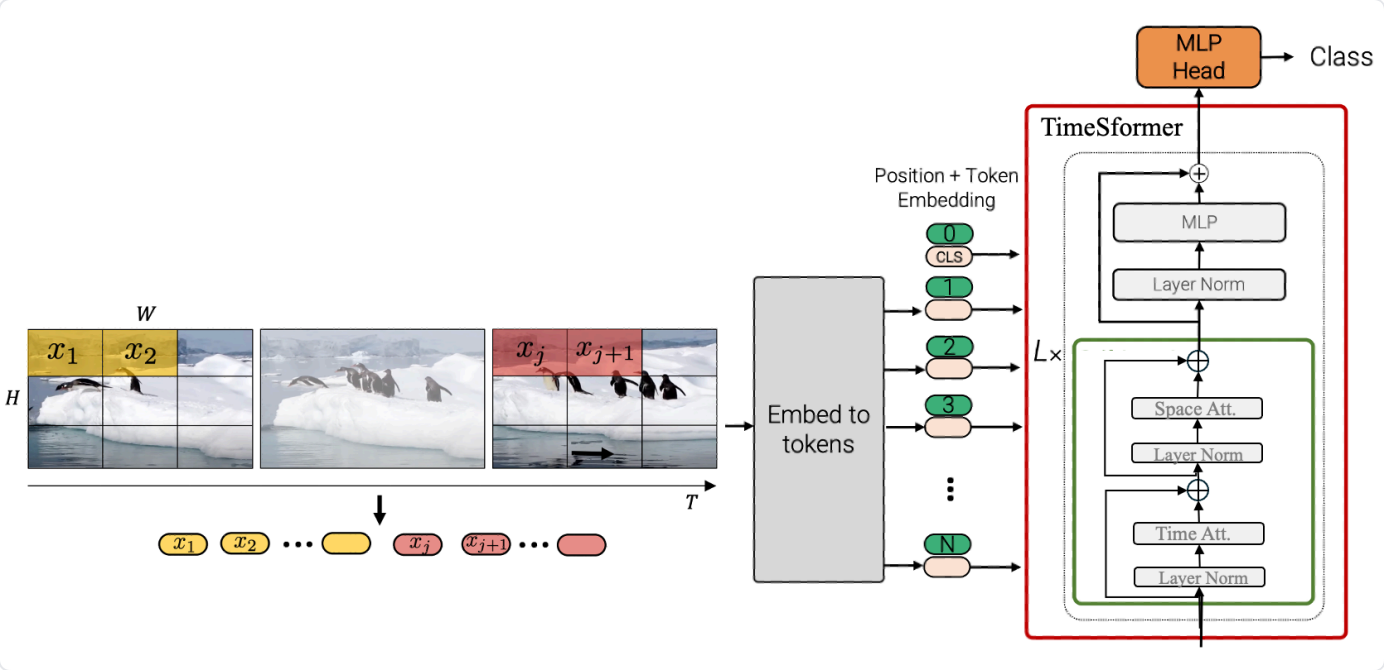
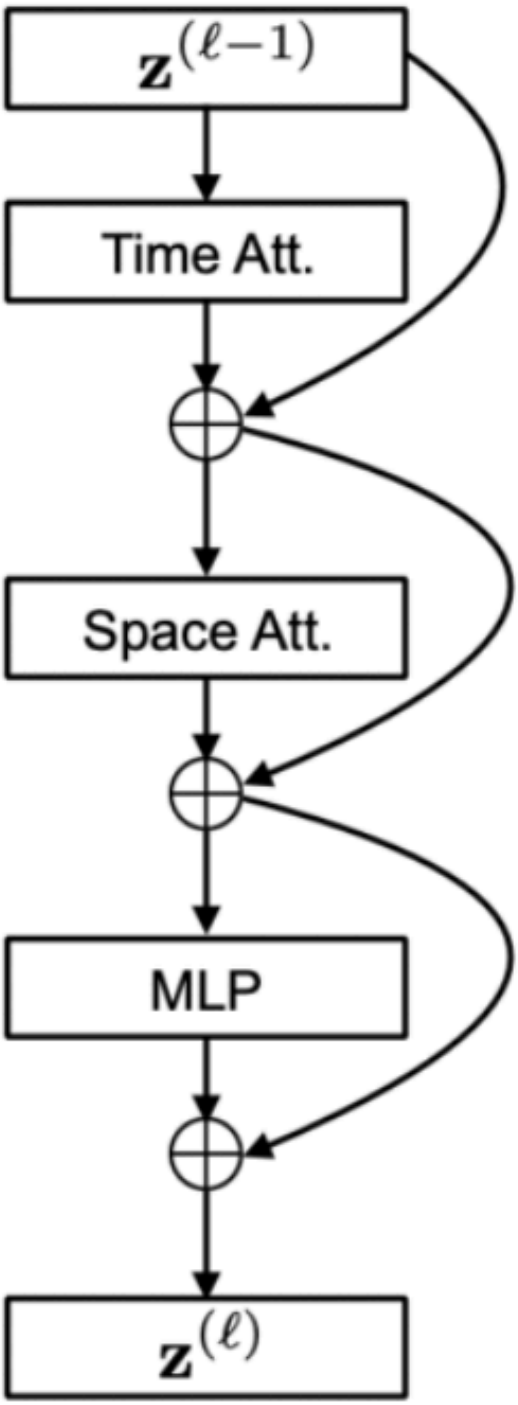
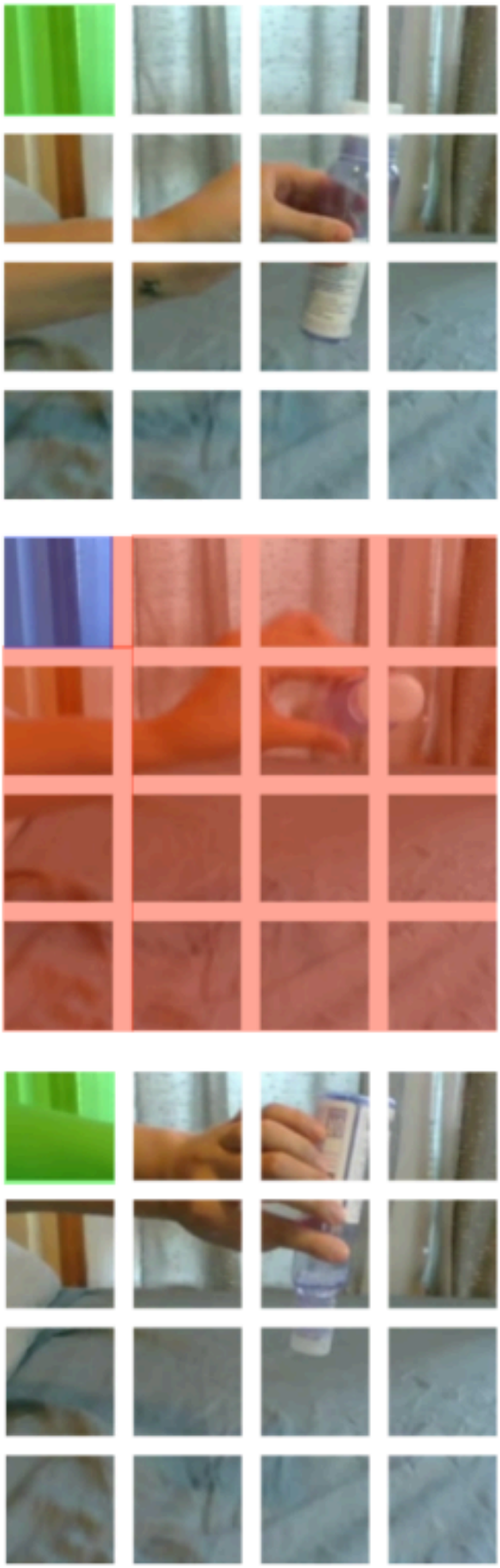


图 2-9 TimeSformer 的模型结构图，先对每一张图像类似 vit 划分为不同的图像块，然后通过 embedding 转换为 token，之后送入到 TimeSformer 中。

- 理论讲解



Divided Space-Time Attention (T+S)



Divided Space-Time Attention (T+S)

图 2-10 左图：TimeSformer 模型的 bolck 块的组成，右图：时空自注意力方案的可视化。每个视频片段被视为由大小为 16 × 16 像素的帧级图像块序列组成。为了说明，我们用蓝色表示查询图像块，并用非蓝色显示在每种方案下其自注意力的时空邻域。没有颜色的图像块不会参与蓝色图像块的自注意力计算。在同一方案中出现的多种颜色表示注意力分别沿不同维度（例如 （T+S） 中的空间和时间维度）独立应用。

- 代码 TimeSformer Block 核心代码，详细代码见：`urbanpipe_video/helper/TimeSformer.py`

```
# ====================================
# TimeSformer Block (Divided Attention)
# ====================================
class TimeSformerBlock(nn.Module):

    def __init__(self, config, num_frames, num_patches):
        super().__init__()

        self.num_frames = num_frames
        self.num_patches = num_patches

        # Spatial attention
        self.spatial_norm = nn.LayerNorm(config.d_model)
        self.spatial_attn = MultiHeadAttention(config)

        # Temporal attention
        self.temporal_norm = nn.LayerNorm(config.d_model)
        self.temporal_attn = MultiHeadAttention(config)

        # FFN
        self.ff_norm = nn.LayerNorm(config.d_model)
        self.ff = FeedForward(config)

    def forward(self, x):
        """
        x shape:
        (B, 1 + F*P, D)
        """
```

```
        B, N, D = x.shape

        cls = x[:, 0:1]     # (B,1,D)
        tokens = x[:, 1:]   # (B,F*P,D)

        F = self.num_frames
        P = self.num_patches

        # reshape
        tokens = tokens.view(B, F, P, D)



        # ====================================
        # 2. Temporal Attention (论文第一步)
        # 每个patch位置 across frame attention
        # ====================================
        temp = tokens.permute(0, 2, 1, 3)  # (B,P,F,D)
        temp = temp.reshape(B*P, F, D)
        temp = temp + self.temporal_attn(
            self.temporal_norm(temp)
        )
        tokens = temp.view(B, P, F, D).permute(0, 2, 1, 3)

        # ====================================
        # 1. Spatial Attention (论文第二步)
        # 每一帧内部做 attention
        # ====================================
        spat = tokens.reshape(B*F, P, D)
        spat = spat + self.spatial_attn(
            self.spatial_norm(spat)
        )
        tokens = spat.view(B, F, P, D)

        # ====================================
        # merge back
        # ====================================
        tokens = tokens.reshape(B, F*P, D)
        x = torch.cat([cls, tokens], dim=1)

        # ====================================
```

```
        # FFN
        # ====================================
        x = x + self.ff(
            self.ff_norm(x)
        )

        return x
```

（2）代码

- 训练代码：`urbanpipe/urbanpipe_video/train.py`

```
python3 ./train.py \
    --base_path ../dataset/tony_data_video/ \
    --label_file_train ../dataset/video_label_file/video_train.json \
    --label_file_val ../dataset/video_label_file/video_val.json \
    --model_name timesformer \
    --learning_rate 0.001 \
    --epochs 10
```

2.6.4、基于超图

在基于图像帧的动作识别方法中，通常将视频级标签直接分配给每一帧进行监督，但这种方式忽略了视频中动作的时间动态和语义连续性，尤其是在长视频中，大量帧可能仅包含背景信息或不完整动作，却仍被赋予完整的视频标签，从而引入标签噪声并导致模型学习到有偏差的特征表示。虽然基于视频序列的方法能够通过显式建模时间关系缓解这一问题，但这类方法通常计算开销较大、训练复杂度较高，并且难以充分利用现有成熟的图像分类模型及其丰富的预训练资源。因此，如何在保持图像模型高效性和良好性能的同时，有效引入时间信息，成为一个值得关注的问题。

受到 ICLR 2022 论文《SIFAR: CAN AN IMAGE CLASSIFIER SUFFICE FOR ACTION RECOGNITION? 》的启发，该方法（如图 2-11）通过重组视频帧构建超图像，将视频的时序信息编码到空间结构中，使图像分类器能够直接用于动作识别任务。基于超图的方法整体结构如图 2-11 所示。具体来说，我们首先从视频中提取帧，并使用均匀采样得到多个帧图像（下图为 9 帧）。接下来，针对每个视频，我们将增强的这是帧图像拼贴成超图像，并将其视为超图像。最后，我们将这些超图像送入图像分类网络进行端到端的训练。

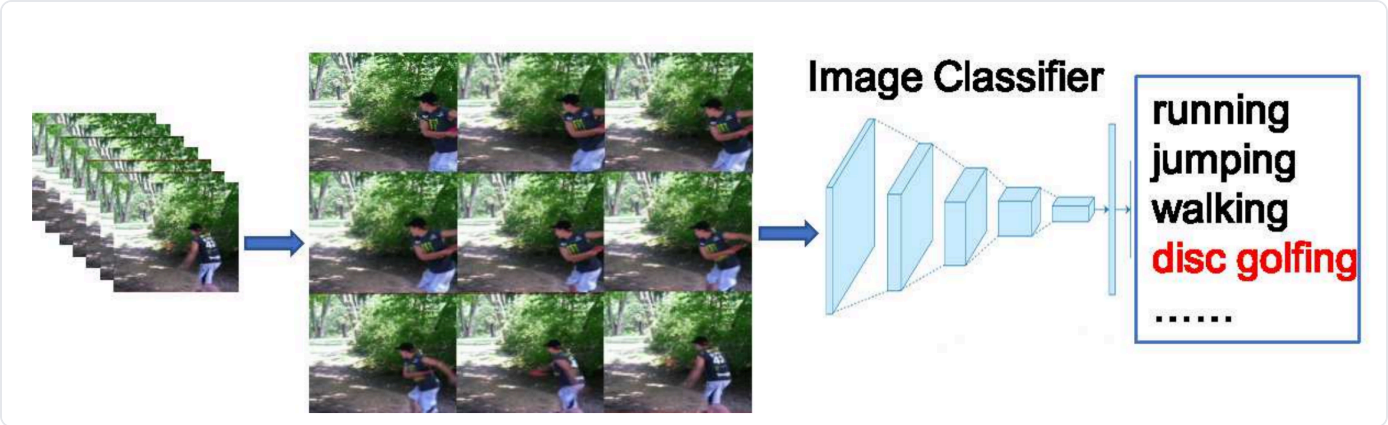
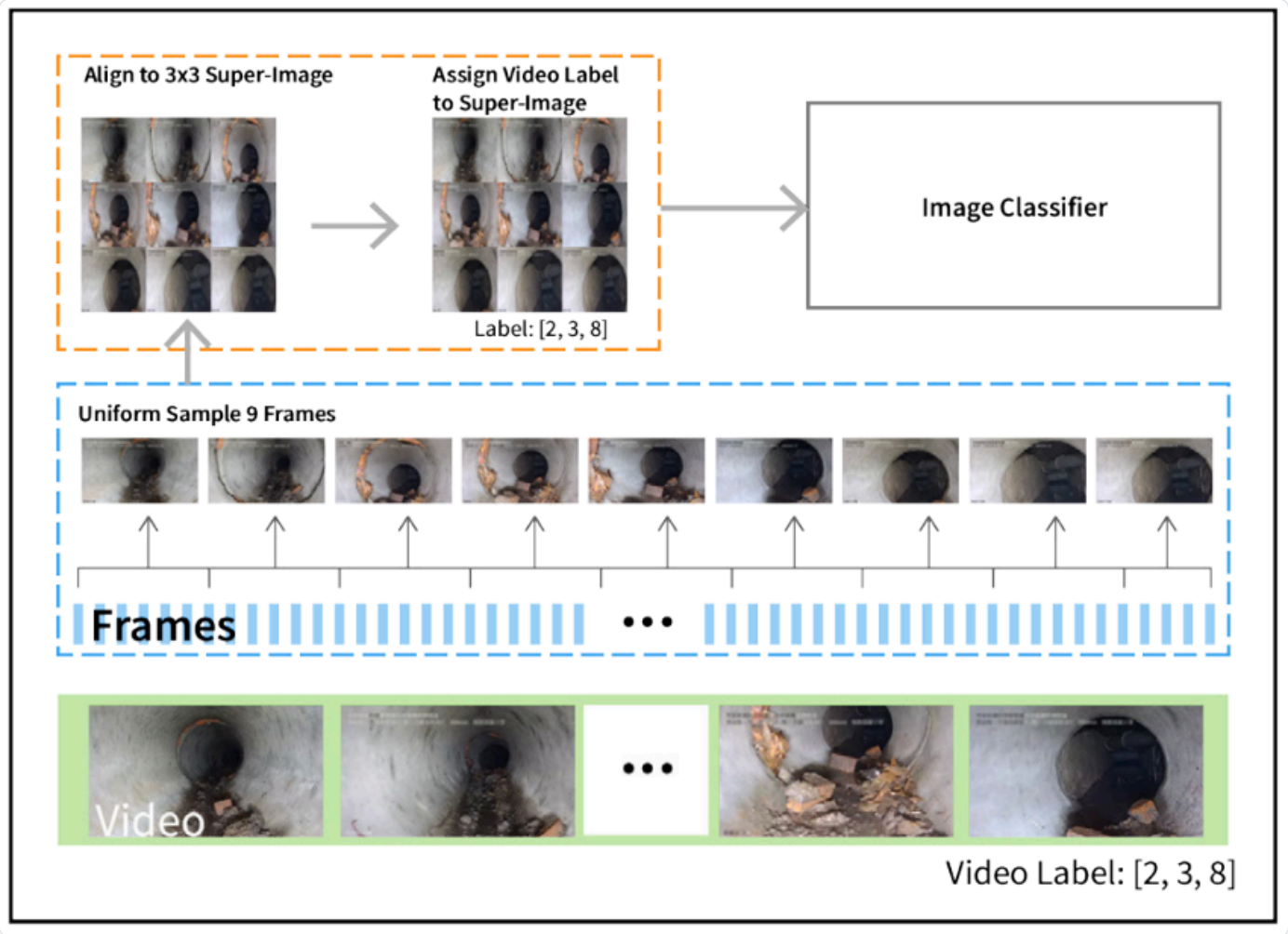


图 2-11 SIFAR 结构示意图：一段输入视频帧序列首先根据 3×3 的空间布局重新排列成一个超图像（super image），然后将该超图像输入到图像分类器中进行识别。

考虑到数据集中的缺陷通常是空间上的细粒度问题，并且与时序关系关系不大，因此我们选择了这种将视频问题转化为超图问题的方法，以更好地处理空间特征。在本项目中，我们借鉴 SIFAR 的这一思想，将视频帧重组为超图像作为模型输入，从而在无需引入复杂时序模型的情况下，使图像分类器能够利用视频中的时序上下文信息，并提升动作识别的准确性和鲁棒性，整体结构示意图如图 2-12 所示。



2-12 基于超图的方法。首先，从视频中提取帧，并使用均匀采样获得多个帧图像；其次，将对采样的帧图像进行数据扩充；第三，对于每个视频，将增强帧图像拼贴为超级图像；最终，将超级图像发送到图像分类网络进行端到端的训练。

（1）模型 vit_h_14

模型等同于基于视频帧，但是输入的尺寸变大了，预训练支持（torch 官方支持的模型权重）大分辨率输入的只有 `models.vit_h_14(weights='DEFAULT')` 输入尺寸大小为\ `518， 518`\ 的。

（2）代码

- 等间隔抽取 9 帧，形成超图

- 代码位置： `urbanpipe_superImage/tools/video2superImage.py`

```
python3 ./tools/video2superImage.py
```

- 训练

- 代码位置：`urbanpipe/urbanpipe_superImage/train.py`

```
python3 ./train.py \
--base_path ./dataset/superImage \
--label_file_train ../dataset/video_label_file/video_train.json \
--label_file_val ../dataset/video_label_file/video_val.json \
--model_name vit_h_14 \
--batch_size 24 \
--learning_rate 0.001 \
--epochs 20
```

- **参数说明**

- `--base_path`：视频帧数据目录 - `--label_file_train`：训练标签 JSON - `--label_file_val`：验证标签 JSON - `--model_name`：模型名称，支持：ResNet 系列：resnet18、resnet34、resnet50 等；ViT 系列：vit_b_16、vit_b_32、vit_l_16 - `--batch_size`: batch size - `--learning_rate`：学习率 - `--epochs`：训练轮数

2.6.5、基于多模态大模型

- 见代码目录 mllm 下
- 基于 Qwen-VL 多模态大模型，将抽取出来的超图+prompt 一块送入 MLLM，然后使用 lora 对 LLM 部分，进行微调，来实现输出。

（1） 任务描述

在管道缺陷数据集中，各类别的样本分布往往呈现严重的**长尾特征**——"正常""沉积"等头部类别样本充裕，而"脱落""错口"等尾部类别样本极为稀缺。传统判别式模型在这些尾部类别上往往表现不佳。基于多模态大模型思路是：借助多模态大模型在海量预训练数据上习得的强泛化能力和丰富的视觉-语义先验知识，弥补尾部类别样本不足的问题，从而显著提升整体分类性能。

（2）问题建模

这是一个典型的**多标签分类**问题：

此处为飞书电子表格（内容见飞书原文）

17 种缺陷类别定义：

此处为飞书电子表格（内容见飞书原文）

在多模态大模型框架下，多标签分类主要有以下两种实现方式：

此处为飞书电子表格（内容见飞书原文）

本项目选择方案：SFT 生成方案

SFT 生成方案的思路是**将多标签分类转化为一个条件文本生成任务**。模型接收图片和 Prompt 作为输入，直接生成一段包含标签信息的结构化 JSON 文本。通过监督微调，模型学会根据图像内容生成正确的标签序列。



选择理由：

1. **长尾分布挑战**：管道缺陷数据集的类别分布严重不均衡，部分尾部类别样本极为稀缺。SFT 方案可以通过 Prompt 注入类别语义信息，借助大模型的预训练先验知识，在少样本类别上获得更好的泛化能力。

（3）数据构造

同一组标签可能有多种表达方式：`{"缺陷": [6, 3, 0]}` — 顺序不同但语义相同，这会让模型产生困惑——同一个输入对应多种"正确"输出，学习信号相互矛盾，降低学习效率。因此，我们按照 label 从小到大进行排序来预处理，确保了**映射的唯一性**：一组缺陷 → 唯一一种表达方式。

- Prompt 设计：本项目的 Prompt 考虑使用 CoT+few-shot 来完成：

- **Chain-of-Thought（CoT，思维链）**：在"分析方法"部分引导模型按步骤思考——依次观察、判断、汇总、去重、排序。虽然我们要求模型"在内部思考，不要输出"以保持输出简洁，但这一思维框架有助于模型组织推理过程，降低遗漏和误判的概率。 - **Few-shot（少样本示例）**：提供 3 个典型示例，分别覆盖"无缺陷""双缺陷""缺陷组合"三种场景。这些示例不仅教会模型正确的输出格式，更重要的是锚定了输出的风格和粒度，减少模型生成不符合预期的输出。

```
SYSTEM_PROMPT = (
    "你是一个管道检测专家，需要根据图像识别管道内部缺陷。\n\n"

    "输入说明：\n"
    "输入图片由同一视频中等间隔抽取的9帧组成，并按时间顺序拼接成一张大图。\n"
    "请同时观察9个子图，检测可能存在的缺陷。\n\n"

    "缺陷类别共有17种：\n"
    "0 正常\n"
    "1 沉积\n"
    "2 腐蚀\n"
    "3 障碍物\n"
    "4 结垢\n"
    "5 树根\n"
    "6 破裂\n"
    "7 变形\n"
    "8 异物穿入\n"
    "9 浮渣\n"
    "10 脱节\n"
    "11 暗接\n"
    "12 残墙\n"
    "13 渗漏\n"
    "14 起伏\n"
```

```
    "15 错口\n"
    "16 脱落\n\n"

    "分析方法，一步一步思考（在内部思考，不要输出）：\n"
    "1. 依次观察9个子图。\n"
    "2. 判断每个子图是否存在缺陷。\n"
    "3. 汇总所有子图中的缺陷类别。\n"
    "4. 去重。\n"
    "5. 按类别编号从小到大排序。\n\n"

    "Few-shot 示例：\n\n"

    "示例1：\n"
    "图像情况：管道内部光滑，没有明显异常。\n"
    "输出：\n"
    '{"缺陷": [0]}\n\n'

    "示例2：\n"
    "图像情况：管道底部有沉积，同时管壁存在腐蚀。\n"
    "输出：\n"
    '{"缺陷": [1, 2]}\n\n'

    "示例3：\n"
    "图像情况：管道出现树根侵入，并伴随沉积。\n"
    "输出：\n"
    '{"缺陷": [1, 5]}\n\n'

    "输出规则：\n"
    "1. 每张图最多输出5种缺陷。\n"
    "2. 如果没有发现缺陷，请返回 {\"缺陷\": [0]}。\n"
    "3. 只允许输出JSON，不要输出解释、文字或代码块。\n"
    "4. 返回格式必须严格如下：\n"
    '{"缺陷": [类别编号1, 类别编号2, ...]}'
)
```

```
def process_func(example):
    """
    多模态数据预处理函数
    """
```

```
MAX_LENGTH = 8192

output_content = example['labels']

# 提取图像路径
file_path = example['image_path']
file_path = os.path.join("./dataset/superImage", file_path)

# 标准化输出标签
normalized_output = normalize_label(output_content)

# 构建多模态对话模板
messages = [{
    "role": "user",
    "content": [
        {
            "type": "image",
            "image": file_path,
            "resized_height": 518,
            "resized_width": 518
        },
        {
            "type": "text",
            "text": SYSTEM_PROMPT
        },
    ]
}]

# 应用模板生成结构化文本
text = processor.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

# 处理视觉输入
image_inputs, video_inputs = process_vision_info(messages)

# 多模态特征整合
inputs = processor(
```

```
        text=[text],
        images=image_inputs,
        videos=video_inputs,
        padding=True,
        return_tensors="pt",
    )

    inputs = {k: v.tolist() for k, v in inputs.items()}

    # 使用标准化后的标签作为输出
    response = tokenizer(normalized_output, add_special_tokens=False)

    # 构建完整输入序列
    input_ids = inputs["input_ids"][0] + response["input_ids"] + [tokenizer.pad_token_id]
    attention_mask = inputs["attention_mask"][0] + response["attention_mask"] + [1]

    # 标签：指令部分掩码为-100
    labels = [-100] * len(inputs["input_ids"][0]) + response["input_ids"] + [tokenizer.pad_toke

    # 长度截断
    if len(input_ids) > MAX_LENGTH:
        input_ids = input_ids[:MAX_LENGTH]
        attention_mask = attention_mask[:MAX_LENGTH]
        labels = labels[:MAX_LENGTH]

    return {
        "input_ids": torch.tensor(input_ids),
        "attention_mask": torch.tensor(attention_mask),
        "labels": torch.tensor(labels),
        "pixel_values": torch.tensor(inputs['pixel_values']),
        "image_grid_thw": torch.tensor(inputs['image_grid_thw']).squeeze(0)
    }
```

（4）模型微调

基于 LoRA 来对 MLLM 中的 LLM 部分，进行微调，vit 部分冻结，具体的 LoRA 参数以及作用的位置如下：

```
processor = AutoProcessor.from_pretrained(MODEL_PATH)

print("[2/5] 加载基础模型...")
model = Qwen2VLForConditionalGeneration.from_pretrained(
    MODEL_PATH,
    device_map="auto",
    torch_dtype=torch.bfloat16,
    trust_remote_code=True,
)
model.enable_input_require_grads()

# ==================== 模型配置阶段 ====================
print("[4/5] 配置LoRA...")

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    target_modules=[
        "q_proj", "k_proj", "v_proj",
        "o_proj", "gate_proj",
        "up_proj", "down_proj"
    ],
    inference_mode=False,
    r=8,
    lora_alpha=32,
    lora_dropout=0.01,
    bias="none",
)

peft_model = get_peft_model(model, lora_config)
```

- 5.2 训练参数详解

```
training_args = TrainingArguments(
        output_dir=OUTPUT_DIR,
        per_device_train_batch_size=8,
        gradient_accumulation_steps=4,
        logging_steps=10,
        num_train_epochs=20,
        save_steps=2,
```

```
        save_total_limit=5,
        learning_rate=1e-4,
        warmup_ratio=0.05,
        lr_scheduler_type="cosine",
        save_on_each_node=True,
        gradient_checkpointing=True,
        report_to="none",
        dataloader_num_workers=4,
        bf16=True,
    )
```

（5）模型推理

```
## 推理流程（Inference Pipeline）


┌──────────────────────┐
│      输入数据                    │
│  Image + Prompt                  │
└──────────┬──────────┘
                  │
                  ▼
┌──────────────────────┐
│     Qwen2-VL 模型                │
│   (Base Model + LoRA)            │
└──────────┬──────────┘
                  │
                  ▼
┌──────────────────────┐
│     JSON 文本生成                 │
│   Greedy Decoding                │
└──────────┬──────────┘
                  │
                  ▼
┌──────────────────────┐
│   JSON 解析与后处理                 │
│ JSON → 正则 → 默认值              │
└──────────┬──────────┘
                  │
```

```
                ▼
┌──────────────────────┐
│     标签 ID 列表                  │
│       [0, 3, 6]                  │
└──────────────────────┘
```

```
# ==================== 推理函数 ====================

def predict(messages, model):
    """
    多模态推理函数

    参数:
        messages: list - 多模态对话数据
        model: PreTrainedModel - 模型实例

    返回:
        str - 生成的文本响应
    """
    # 构建对话模板
    text = processor.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    # 处理视觉输入
    image_inputs, video_inputs = process_vision_info(messages)

    # 多模态特征整合
    inputs = processor(
        text=[text],
        images=image_inputs,
        videos=video_inputs,
        padding=True,
        return_tensors="pt",
    ).to("cuda")

    # 生成响应
```

```
    generated_ids = model.generate(
        **inputs,
        max_new_tokens=64,
        do_sample=False,
    )

    # 去除输入部分
    generated_ids_trimmed = [
        out_ids[len(in_ids):]
        for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
    ]

    # 解码
    output_text = processor.batch_decode(
        generated_ids_trimmed,
        skip_special_tokens=True,
        clean_up_tokenization_spaces=False
    )[0]

    return output_text
```

为什么分类任务必须用贪心解码?

- **生成式任务**（写作、对话）：需要 \`do_sample=True\` 以引入随机性，增加输出的多样性和创造性。
- **判别式任务**（分类、信息抽取）：需要 \`do_sample=False\`，确保同一输入始终得到同一输出。分类任务要求的是**确定性和一致性**，而非多样性。

（6）评估指标

mAP 是多标签分类中:

```
mAP = (AP_类别1 + AP_类别2 + ... + AP_类别N) / N

其中每个类别的AP通过其P-R曲线下面积计算
```

（7）总结

```
## 训练流程

原始数据                    模型准备                    训练
  │                          │                          │
  ▼                          ▼                          ▼
标签标准化                Qwen2-VL-2B               LoRA 配置
(升序 / 去重 / 校验)       (加载预训练权重)           (r = 8, α = 32)
  │                          │                          │
  ▼                          ▼                          ▼
构建对话模板              冻结基础参数               SFT 训练
(图像 + Prompt)           (仅训练 LoRA 适配器)       (Cross-Entropy Loss)
  │                          │                          │
  ▼                          ▼                          ▼
编码为 Tensor             注入 LoRA 到               保存 Checkpoint
(input_ids,               7 类线性层
 labels, pixel_values)

## 推理与评估

推理                        评估
  │                          │
  ▼                          ▼
加载 Base + LoRA          mAP 计算
  │                          │
  ▼                          ▼
贪心解码生成 JSON          基于 PR 曲线
  │                          │
  ▼                          ▼
多层容错解析               计算 AP
(JSON → 正则 → 默认值)     (Per-class AP)
  │                          │
  ▼                          ▼
输出标签列表               计算最终指标
[0, 3, 6]                  ★ mAP
```

2.7、优化（可选）

2.7.1、多模型融合

在机器学习实践中，不同的模型往往对数据的特定方面具有独特的捕捉能力。例如，**视频帧模型：** 专注于静态空间特征，擅长识别视频中的物体外观、场景背景等细节。**基于视频的模型：** 专注于时序动态，能够捕捉动作的连贯性、物体运动轨迹等关键时间信息。**基于超图的模型：** 专注于高阶关联，通过建模实体间的复杂交互关系，挖掘深层语义联系。**基于多模态的大模型：**对于小样本，难样本，具有较好的泛化能力。

（1）预测分数的加权和方法

本项目采用预测分数的加权和方法进行模型融合。这种方法的核心在于为每个模型的预测结果分配一个权重，然后将这些加权后的预测分数进行求和，最终得到融合后的预测分数。具体步骤如下：

1. **独立模型训练与预测：**

首先，训练多个独立的模型，例如视频帧、基于视频的、基于超图的模型。在多标签分类场景中，每个模型通常会输出一个针对每个类别的预测分数（例如，通过全连接层后接 Sigmoid 激活函数，输出 17 个类别的分数，每个分数介于 0 到 1 之间，表示样本属于该类别的概率）。

1. **权重分配：**

根据每个模型在验证集上的性能，为其分配一个相应的权重。根据在验证集上的表现，性能表现越好的模型，会获得越大的权重。

1. **加权求和：**

对于每一个待预测的样本，将其输入到所有已训练好的模型中，获取每个模型对所有类别的预测分数。 假设我们有 N 个模型，每个模型 M_i 对于第 j 个类别的预测分数是 $P_{i,j}$，且对应的权重是 W_i。那么，融合后的模型对第 j 个类别的预测分数 $P_{fusion,j}$ 可以表示为：

$$P_{fusion,j} = \sum_{i=1}^{N} (W_i \times P_{i,j})$$

1. 其中，所有权重的和通常被归一化为 1，即 $\sum_{i=1}^{N} W_i = 1$。
2. 最终预测：

在获得融合后的预测分数 $P_{fusion,j}$ 后，来作为最终的分数，用来计算 mAP。

通过加权求和的方式，模型融合能够充分利用各模型的优势，在多标签分类任务中提供更为稳健和准确的预测结果，从而提升整体项目的性能。

（2）三模型加权融合案例

假设我们要识别一段视频中包含的动作标签（共 17 类）。我们有三个独立的模型，它们在验证集上的表现（如 mAP）决定了它们的**权重分配**：

此处为飞书电子表格（内容见飞书原文）

注：权重之和 $0.2 + 0.5 + 0.3 = 1.0$。

- 1. 单模型预测

对于某一个特定的样本，三个模型分别对**第 j个类别**（假设是“漏水”）给出了置信度分数：

- M_1 **预测分数 （ $P_{1,j}$）：**0.40 （比较犹豫）
- M_2 **预测分数 （ $P_{2,j}$）：**0.90 （非常确定在漏水）
- M_3 **预测分数 （ $P_{3,j}$）：**0.70 （认为概率较大）

- 2. 融合计算 （Weighted Sum）

我们将每个模型的分数乘以其对应的权重，然后求和得到最终得分$P_{fusion,j}$：

$$P_{fusion,j} = (0.2 \times 0.40) + (0.5 \times 0.90) + (0.3 \times 0.70)$$ = **0.74**

- 3. 结果分析

在这个例子中，虽然视频帧模型 （M_1） 给出的分数较低（可能是因为画面模糊），但由于视频模型 （M_2） 表现更稳健且权重最高，它成功地“修正”了整体判断。融合后的分数 **0.74** 综合了三个视角的观察：

1. **容错性：** 如果 M_1 因为噪声产生错误的高分，较低的权重（0.2）会削弱其对最终结果的负面影响。
2. **稳健性：** 当 M_2 和 M_3 同时给出高分时，最终结果会非常可靠。

三、简历怎么写?

面向真实场景的细粒度城市管道视频缺陷检测 ECCV 挑战赛

- 任务描述：给定一段城市管道的视频，识别可能存在的多种缺陷，例如腐蚀、沉积等。
- 实现方法：1、**视频帧方案**：将视频理解问题转换为图片理解问题，然后把图片的结果融合为视频的结果，使用 Vision Transformer Base 模型。 2、**视频理解方案**：针对视频帧缺失时序信息的问题，采用 TimeSformer 等视频理解模型来进行缺陷检测。 3、**超图方案**：鉴于视频理解引入的噪声较多，以及一些缺陷（比如沉积）和时序无关，采用将 9 张图片构建为超图的方案，之后作为图片理解的方案进行，主要采用 Vision Transformer Huge 图片理解模型。 4、**基于多模态大模型方案：**利用多模态大模型对视频关键帧或超图图像进行理解，并结合缺陷类别的文本描述进行多标签预测。多模态大模型具备较强的语义理解和泛化能力，能够在小样本和长尾分布条件下提升缺陷识别效果。
- 技术难点 1、**存在多种缺陷类别问题**：采用多个二值损失函数来替代多分类损失函数以解决该问题。 2、**不同缺陷类别的视频关注的信息不同**：采用多模型融合的方案，将上述四种方案进行融合，从而得到最终的解决方案。
- 取得结果：在挑战赛上取得了 64.2mAP 的成绩。

项目名称：面向真实场景的细粒度城市管道视频缺陷检测（ECCV Challenge）

任务描述：针对真实城市管道巡检视频中的复杂场景，识别可能存在的多种细粒度缺陷（如腐蚀、沉积、裂纹等）。

实现方法：1、多维特征建模：结合 ViT-Base 图像方案与 TimeSformer 视频理解方案，兼顾静态空间细节与动态时序特征。2、超图全局表征：将 9 帧关键图像构建为 3×3 超图，利用 ViT-Huge 进行高分辨率处理，在减少时序噪声的同时强化空间全局感知。3、多模态大模型：利用 MLLM 的语义先验，通过文本 Prompt 引导，进一步提升模型对小样本缺陷的泛化能力。4、优化：采用 BCE 损失处理多标签问题，并对多模型结果进行加权融合。

取得结果：在验证集上取得 64.2 mAP ，在真实工业管道巡检场景中具有较高的实际应用价值。

（推荐）面向真实场景的细粒度城市管道视频缺陷检测（ECCV Challenge）

任务描述：针对真实城市管道巡检视频中的复杂场景，识别可能存在的多种细粒度缺陷（如腐蚀、沉积、裂纹等）。

实现方法：1、多维特征建模：结合 ViT-Base 图像方案与 TimeSformer 视频理解方案，兼顾静态空间细节与动态时序特征。2、超图全局表征：选取 9 帧图像构建 3×3 超图结构，利用 ViT-Huge 进行高分辨率处理，增强全局空间感知并减少时序噪声。3、多模态大模型：利用 MLLM 的语义先验，通过 Prompt 引导提升模型对小样本缺陷的泛化能力。4、优化：采用 BCE 损失处理多标签问题，并对多模型结果进行加权融合。

取得结果：在验证集上取得 64.2 mAP ，在真实工业管道巡检场景中具有较高的实际应用价值。

四、面经

深度学习基础面经（重要）+项目1（了解，dropout L2）+项目2（了解，残差链接）+项目3（比较重要 bert, multi head attention, ffn) + 项目4（LLM相关 lora、解码、prompt设计）；前面项目的具体数据、项目强绑定的问题不用看。

一、背景与任务理解

1. 请简要介绍一下这个项目（比赛）的背景和任务定义？

城市管道系统是城市基础设施的重要组成部分，但由于管道大多位于地下，运行状态难以直接观察，一旦发生缺陷往往难以及时发现。本项目来源于 ECCV 竞赛体系中的 UrbanPipe 挑战赛，基于真实城市管道检测视频数据，要求构建模型从管道检测视频中识别缺陷类型。

任务定义： 这是一个**多标签分类问题**。给定一段管道检测视频，需要预测其中存在的所有缺陷类别。与单标签分类不同，一个视频中可能同时存在多种缺陷，例如同一段视频中既可能出现结构变形，也可能同时存在裂缝或沉积。数据集包含 1 个正常类别和 16 个缺陷类别，共 17 类，每个视频被 1 到 5 个类别标注，平均 1.4 个标签。

2. 简单介绍一下你的技术方案？请用一分钟简要介绍你的整个项目？

该项目针对管道视频缺陷检测任务，采用了四种互补的建模方案，并通过加权融合得到最终结果：

- **视频帧方案：** 将视频均匀采样为 16 帧，每帧独立送入图像分类网络（ViT-Base/16）进行预测，最终将所有帧的预测概率取平均作为视频级结果。
- **视频理解方案：** 使用 TimeSformer 模型直接以视频片段为输入，通过分离的时间注意力和空间注意力同时建模时空特征。
- **超图方案：** 从视频中均匀采样 9 帧，按 3×3 布局拼接成一张超图像，送入 ViT-H/14 进行分类，在保持图像模型效率的同时融入时间信息。
- **多模态大模型方案：** 基于 Qwen2-VL，将超图图像结合精心设计的 Prompt（包含 CoT 和 Few-shot），通过 LoRA 微调使大模型输出结构化的缺陷标签 JSON。

在损失函数方面，采用 BCEWithLogitsLoss 对每个类别独立建模。最终通过加权融合四种方案的预测结果，取得了 64.2 mAP 的成绩。

---

3. 你觉得管道缺陷检测和普通的图像分类任务相比，最大的难点是什么？

主要有三个难点：

1. **多标签问题：** 普通图像分类是单标签互斥分类，而管道缺陷检测中一个视频可能同时存在多种缺陷，需要对每个类别独立判断，不能简单使用 softmax。
2. **长尾分布：** 数据集呈现严重的长尾分布，"正常""沉积"等类别样本充裕，而"脱落""错口"等尾部类别样本极为稀缺，模型容易偏向头部类别。
3. **细粒度特征：** 不同缺陷依赖的信息维度不同——有些缺陷主要体现在空间结构上（如变形），有些则需要时间维度的动态信息，有些缺陷之间视觉差异非常微小，识别难度大。

---

4. 是否参加比赛？你的 64.2 mAP 在比赛中排名如何？

没有直接参加比赛，但使用的是比赛的公开数据集 UrbanPipe 进行研究和实践。数据集依然可以从 OpenDataLab 网站下载。64.2 mAP 是我在该数据集上复现和优化后取得的成绩。

---

5. 前几名的方案和你有什么不同？

前几名的方案通常在以下几个方面做得更深入：

1. **更大规模的预训练模型：** 可能使用了更大的 ViT 模型或专门针对视频预训练的模型（如 VideoMAE、InternVideo 等）。
2. **更精细的长尾问题处理：** 如类别重采样、Focal Loss、类别平衡损失等专门的长尾学习策略。
3. **更多的模型多样性：** 可能融合了更多不同架构的模型来增加互补性。

官网上可以看到前三名的方案：官网地址

- **Track 5 - UrbanPipe Challenge on Fine-grained Video Anomaly Recognition**

In this challenge, we have received 1393 valid submissions for final evaluation. Winner teams' performance and reports can be found below.

此处为飞书电子表格（内容见飞书原文）

二、数据相关

1. 数据集的规模是多少？训练集和验证集是怎么划分的？划分比例是多少？

UrbanPipe 数据集共包含 **9600 个视频**，总时长超过 55 小时。每个视频持续时间从 0.7 秒到 177.4 秒不等，平均为 20.7 秒。训练集和验证集通过 JSON 标签文件（video_train.json 和 video_val.json）进行划分，通常按照 **8:2** 的比例划分训练集和验证集。

考虑到我们算力有限，选择了 3500 个视频作为训练集、500 个视频作为验证集、500 个视频作为测试集。

---

2. 数据集的类别和标签分布信息？

数据集包含 **17 个类别**（1 个正常 + 16 个缺陷），数据呈现**自然的长尾分布**，正常、沉积、腐蚀等头部类别样本较多，脱落、错口等尾部类别样本极少。每个视频被 1 到 5 个类别标注，平均 1.4 个标签。

3. 基于超图的方法多少张图？如何采样？

基于超图的方法从视频中**均匀采样 9 帧**。采样方式为等间隔采样，即将视频的总帧数等分为 9 段，每段取一帧。然后将这 9 帧按照 **3×3 的网格布局**拼接成一张超图像，作为图像分类网络的输入。

---

4. 基于视频的方法，采样策略？采样多少帧？

基于视频的方法采用**均匀采样策略，从完整视频中等间隔提取 8 帧**。具体做法是将视频总帧数等分为 8 段，每段取中间帧或随机帧。这 8 帧作为一个连续片段整体输入 TimeSformer 模型，模型同时对空间和时间维度进行建模。

---

5. 基于 MLLM 的方法，输入是什么？如何得到？

基于 MLLM 的方法输入是**超图图像 + 文本 Prompt**。

- **图像输入：** 从视频中等间隔抽取 9 帧（或使用之前生成的超图），按时间顺序拼接成一张大图，分辨率调整为 518×518。
- **文本输入：** 精心设计的 System Prompt，包含任务描述、17 种缺陷类别定义、CoT 思维链引导（依次观察→判断→汇总→去重→排序）、3 个 Few-shot 示例以及严格的输出格式要求。
- 图像和文本通过 Qwen2-VL 的处理器转换为多模态 token 序列，一起送入模型。

三、模型架构相关

基于图片

1. 请详细介绍一下 Vision Transformer 的工作原理？

Vision Transformer（ViT）的核心思想是将 Transformer 架构从 NLP 迁移到计算机视觉：

1. **图像分块（Patch Embedding）：** 将输入图像（如 224×224）划分为固定大小的图像块（如 16×16），每个图像块被展平后通过线性投影映射为一个 token 向量。例如 224×224 的图像被划分为 14×14=196 个 patch。
2. **位置编码：** 为每个 patch token 添加可学习的位置编码，以保留空间位置信息。同时在序列开头添加一个可学习的[CLS] token 用于分类。
3. **Transformer 编码器：** 将这 197 个 token（196 个 patch + 1 个 CLS）送入多层 Transformer 编码器。每层包含：

- **多头自注意力（MHSA）：** 让每个 patch 能够关注图像中所有其他 patch，建立全局依赖关系。 - **前馈网络（FFN）：** 两层 MLP，中间用 GELU 激活。 - **Layer Normalization 和残差连接。**

1. **分类头：** 取最后一层的[CLS] token 表示，通过线性层输出类别预测。

ViT 的优势在于全局感受野和强大的特征表示能力，但需要大量数据或预训练才能取得好的效果。

---

2. 基于图片怎么融合的?

在推理阶段，对视频等间隔采样的 16 帧分别送入图像分类网络，每帧得到 17 个类别的预测概率分布。然后在**时间维度上对所有帧的预测概率取平均**，即对 16 帧的每个类别概率值求均值，得到视频级别的最终预测结果。

公式表示： $P_{video,j} = \frac{1}{16} \sum_{i=1}^{16} P_{frame_i,j}$ ，其中 j 为类别索引。

---

3. 介绍一下 ResNet，以及残差块思想？（了解）

ResNet（残差网络） 是何恺明等人在 2015 年提出的深度卷积神经网络架构。

核心思想——残差学习： 传统深层网络存在退化问题，即网络层数增加后性能反而下降。ResNet 通过引入残差**连接（shortcut connection）** 解决这一问题。

残差块结构：

- 输入 x经过两层（或三层）卷积后得到$F(x)$
- 最终输出为 $H(x) = F(x) + x$，即将输入直接加到卷积输出上
- 这样网络只需要学习残差 $F(x) = H(x) - x$，而不是完整映射$H(x)$

优势：

1. 梯度可以通过 shortcut 直接回传，缓解梯度消失问题
2. 使训练极深的网络（如 152 层）成为可能
3. 恒等映射使得增加层数至少不会降低性能

基于超图

4. 超图输入是什么？怎么得到的？分辨率是多少？

超图输入是一张由多帧视频帧拼接而成的大图像。

生成过程：

1. 从原始视频中等间隔均匀采样 9 帧
2. 将这 9 帧按照 3×3 的网格布局拼接成一张大图
3. 最终超图像的分辨率为 **518×518**（配合 ViT-H/14 模型的输入要求）

这种方式将视频的时序信息编码到了空间结构中，使得图像分类模型能够同时观察到多个时间点的管道状态。

---

5. 具体 Vision Transformer 是什么型号？超图方案使用的 ViT-H/14 要求 518×518 的输入，每个子图的实际分辨率是多少？这个分辨率够用吗？

使用的是 **ViT-H/14**（Vision Transformer Huge，patch size 为 14），这是 PyTorch 官方 torchvision 支持的预训练模型中支持大分辨率输入的模型，预训练权重支持 518×518 的输入分辨率。

每个子图的实际分辨率为 **518/3 ≈ 172×172 像素**。

是否够用： 对于本任务基本够用，原因如下：

1. 管道缺陷（如腐蚀、沉积、变形等）通常是较为明显的结构性特征，172×172 的分辨率足以捕捉这些特征。
2. ViT-H/14 的 patch size 为 14，每个子图包含约 12×12=144 个 patch token，提供了足够的空间细节。

3. 实验结果表明超图方案取得了不错的效果，验证了这个分辨率的有效性。
4. 当然，如果需要检测更细粒度的缺陷（如微小裂缝），可以考虑提高分辨率或减少拼接帧数（如 2×2 拼接 4 帧），但这需要在时间覆盖和空间分辨率之间做权衡。

基于视频

6. 请介绍一下 TimeSformer 的思想？（重要）

TimeSformer（Time-Space Transformer） 是 Facebook AI 在 2021 年提出的纯 Transformer 视频理解模型，核心思想是将自注意力机制从图像扩展到视频的时空维度。

设计——分离式时空注意力：

与直接在所有时空 token 上做全局注意力不同（计算量太大），TimeSformer 将注意力分解为两步：

1. **时间注意力（Temporal Attention）：** 对于每个空间位置的 patch，跨所有帧计算注意力。即同一空间位置在不同时间帧之间做 attention，捕捉该位置随时间的变化。
2. **空间注意力（Spatial Attention）：** 在每一帧内部，所有 patch 之间计算注意力，捕捉单帧图像的空间结构特征。

具体流程：

1. 将视频的每一帧按 ViT 的方式划分为 16×16 的 patch，通过 embedding 转换为 token
2. 在每个 TimeSformer Block 中，先做时间注意力，再做空间注意力，最后通过 FFN
3. 这种分离式设计将计算复杂度从 $O((T \times P)^2)$降低到 $O(T \times P^2 + P \times T^2)$，大大提高了效率。

优势： 不需要 3D 卷积，纯 Transformer 架构，能有效建模长距离时空依赖关系，且计算效率优于联合时空注意力。

基于 MLLM

7. Qwen2-VL 的整体架构是什么样的？

Qwen2-VL 是阿里云推出的多模态大语言模型，整体架构由三个组件构成：

1. **视觉编码器（Vision Encoder）：** 基于 ViT 架构，将输入图像划分为 patch 并编码为视觉 token 序列。支持动态分辨率输入。
2. **视觉-语言适配器（Adapter/Projector）：** 将视觉编码器输出的视觉特征投影到与语言模型相同的特征空间，使视觉 token 和文本 token 能够在统一空间中交互。
3. **大语言模型（LLM）：** 基于 Qwen2 的 Transformer decoder 架构，接收拼接后的多模态 token 序列（视觉 token + 文本 token），通过自回归方式生成文本输出。

工作流程： 图像 → ViT 编码 → 适配器投影 → 与文本 token 拼接 → LLM 自回归生成 → 输出文本。

---

8. MLLM 具体用的什么？你用的是 Qwen2-VL-2B，为什么不用更大的模型?

使用的是 **Qwen2-VL-2B**。选择 2B 参数量的模型主要基于以下考虑：

1. **计算资源限制：** 更大的模型（如 7B、72B）需要更多的 GPU 显存和更长的训练时间，2B 模型在单卡或少量 GPU 上即可完成微调。
2. **训练效率：** 2B 模型配合 LoRA 微调，训练速度较快，便于快速迭代实验和调参。
3. **性能足够：** 对于本任务，2B 模型经过 LoRA 微调后已经能够达到不错的效果，尤其在长尾类别上的泛化能力已经体现出优势。
4. **融合策略：** 该方案作为融合中的一个组件，不需要单模型达到极致性能，更注重与其他模型的互补性。

---

9. 多模态大模型中，图像 token 和文本 token 是怎么拼接在一起的?

在 Qwen2-VL 中，多模态 token 的拼接流程如下：

1. **图像处理：** 输入图像经过视觉编码器（ViT）编码为一系列视觉 token，再通过适配器投影到语言模型的 embedding 空间。
2. **文本处理：** 文本 Prompt 经过 tokenizer 编码为文本 token 序列。
3. **序列拼接：** 按照对话模板的格式，将视觉 token 和文本 token 按顺序拼接为一个统一的 token 序列。典型的排列顺序为：`[System Prompt tokens][图像视觉tokens][User指令文本tokens][生成标记]`。

4. **统一处理：** 拼接后的完整序列送入 LLM 的 Transformer decoder，模型通过自注意力机制同时处理视觉和文本信息，实现跨模态的理解和推理。

在训练时，只对生成部分（即标签 JSON）计算交叉熵损失，输入部分的标签被设为 -100 进行掩码。

---

10. Qwen-VL 怎么让它做分类任务?

将分类任务**转化为条件文本生成任务（SFT 生成方案）**：

1. **输入构造：** 将管道图像和包含类别定义的 Prompt 一起输入模型。
2. **输出定义：** 要求模型生成结构化的 JSON 格式文本，如 `{"缺陷": [1, 2]}`，其中数字对应缺陷类别编号。
3. **监督微调（SFT）：** 使用训练数据中的图像-标签对，将标签转换为标准化的 JSON 文本作为目标输出，通过交叉熵损失对模型进行微调。
4. **标签标准化：** 为确保映射唯一性，将标签按编号从小到大排序、去重，避免同一标签集合有多种表达方式。
5. **推理时解析：** 模型生成的 JSON 文本通过 JSON 解析（带正则表达式容错）转换回标签 ID 列表。

这种方式利用了大模型的语义理解能力，不需要额外设计分类头。

---

11. 视觉编码器有没有一起微调？为什么?

没有微调视觉编码器，只冻结了视觉编码器，仅通过 LoRA 微调 LLM 部分。

原因如下：

1. **预训练质量：** Qwen2-VL 的视觉编码器在大规模图像-文本数据上预训练，已经具备强大的视觉特征提取能力，能够有效编码管道图像中的缺陷特征。
2. **防止过拟合：** 管道缺陷数据集规模相对有限（3500 个视频），如果同时微调视觉编码器，参数量过大容易导致过拟合，尤其是在长尾类别上。
3. **训练效率：** 冻结视觉编码器可以大幅减少可训练参数量和显存占用，加快训练速度。
4. **LoRA 的高效性：** 通过 LoRA 仅微调 LLM 中的 7 类线性层（q_proj、k_proj、v_proj、o_proj、gate_proj、up_proj、down_proj），就足以让模型适应特定任务，参数量增加极少

（r=8）。

---

12. 多模态大模型方案输入的是超图还是单帧?

输入的是**超图**。具体来说，从视频中等间隔抽取 9 帧，按时间顺序拼接成一张大图，调整分辨率为 518×518 后输入模型。使用超图而非单帧的原因是：一张超图可以让模型同时观察多个时间点的管道状态，提供更全面的信息用于缺陷判断。

---

13. 你的 Prompt 是怎么设计的？为什么?

Prompt 设计采用了 **CoT（思维链）+ Few-shot（少样本示例）** 的策略：

结构组成：

1. **角色定义：** "你是一个管道检测专家"——锚定模型的专业角色。
2. **输入说明：** 解释图片是由 4 帧拼接而成，引导模型观察所有子图。
3. **类别定义：** 列出 17 种缺陷类别及其编号，提供完整的类别语义信息。
4. **CoT 引导：** 按步骤引导模型思考——依次观察→判断→汇总→去重→排序，降低遗漏和误判概率。
5. **Few-shot 示例：** 3 个典型示例覆盖无缺陷、双缺陷、缺陷组合场景，锚定输出格式和风格。
6. **输出规则：** 严格限定输出为 JSON 格式，限制最多 5 种缺陷，确保输出可解析。

设计原因：

- CoT 帮助模型有条理地推理，减少多标签场景下的遗漏
- Few-shot 示例确保输出格式一致性，减少格式解析错误
- 类别语义描述帮助模型理解缺陷含义，提升长尾类别的识别能力

四种方案的对比与分析

14. 四种方案各自的优缺点是什么?

方案	优点	缺点	输入数据形式	使用的模型	输出	mAP	融合权重
视频帧方案	实现简单；可利用成熟的图像分类模型和丰富的预训练资源；训练成本低	忽略帧间时间关系；对依赖时间变化的缺陷识别能力有限	224✕224	ViT base	17个类别的概率	55.21	0.1
视频理解方案	同时建模时空特征；能捕捉动态变化	计算成本高；本任务中时序信息贡献较小，视频中噪声反而可能干扰判断	8✕224✕224	TimeSformer	17个类别的概率	59.87	0.3
超图方案	在计算效率和	子图分辨率降低（每张约172×172）；	518✕518	ViT Huge	17个类	61.21	0.4

	时间信息利用间取得平衡；特别适合空间结构类缺陷；可直接利用图像预训练模型	帧间的时序关系被弱化为空间布局关系			别的概率		
MLLM方案	泛化能力强；在长尾类别和小样本上表现好；可注入类别语义先验知识	推理速度慢；输出为离散标签不便于与其他模型的连续概率融合；训练资源需求大	518✕518+prompt	Qwen2-VL 2B	缺陷的类别	58.21	0.2

15. 模型怎么融合的?

采用**预测分数加权求和**的方法进行模型融合:

1. **独立训练与预测:** 四种方案分别独立训练,推理时每个模型输出 17 个类别的预测分数(概率值)。
2. **权重分配:** 根据每个模型在验证集上的 mAP 表现分配权重,性能越好的模型权重越大,所有权重之和归一化为 1。
3. **加权求和:** 对于每个待预测样本的第 j 个类别,融合分数为:$P_{fusion,j} = \sum_{i=1}^{N}(W_i \times P_{i,j})$
4. **最终结果:** 融合后的分数直接用于计算 mAP 评价指标。

---

17. 融合后相比最佳单模型提升了多少个点?

融合后相比最佳单模型通常可以提升 **2-4 个 mAP 点**。融合的价值在于:不同模型关注的信息维度不同(空间/时序/语义),互补性使得融合结果更稳健,尤其在尾部类别上,大模型方案的泛化能力可以有效弥补判别式模型的不足。

---

18. 多模态大模型输出的是离散标签,怎么和其他模型的连续概率值进行融合?

这确实是一个挑战。主要有以下几种处理方式:

1. **硬标签转概率:** 将 MLLM 输出的离散标签转换为 0/1 的伪概率(存在该缺陷为 1,否则为 0),然后与其他

模型的连续概率值进行加权融合。这种方式简单直接,但损失了置信度信息。

1. **多次采样估计概率:** 使用 `do_sample=True` 进行多次推理(如 10 次),统计每个类别在多次输出中出现的频率作为概率估计。例如某类别在 10 次中出现 7 次,则概率为 0.7。但这会大幅增加推理时间。
2. **分离融合策略:** 对 MLLM 方案单独处理——只在其他三个模型不确定(概率在 0.3-0.7 之间)的样本上,用 MLLM 的结果作为参考进行修正,而非简单加权。

本项目采用的方式: 采用第 1 种方式,将 MLLM 输出的离散标签转为 0/1 伪概率后参与加权融合,MLLM 方案的权重相对较低,主要发挥对长尾类别的补充作用。

四、训练相关

损失函数和优化器

1. 损失函数是什么?

使用 **BCEWithLogitsLoss（带 Logits 的二元交叉熵损失）**。

选择原因： 本任务是多标签分类问题，每个样本可能同时属于多个类别，各类别之间相互独立。

工作原理：

- 将多标签分类视为 17 个独立的二分类问题
- 对每个类别，模型输出一个 logit 值，经过 sigmoid 函数映射到[0，1]区间作为概率
- 对每个类别分别计算二元交叉熵损失
- 最终损失为所有类别损失的平均值

公式：

$$\mathcal{L} = -\frac{1}{C}\sum_{c=1}^{C}\left(y_c \log \sigma(z_c) + (1-y_c)\log(1-\sigma(z_c))\right)$$

与交叉熵损失的区别： 交叉熵损失使用 softmax，要求所有类别概率和为 1（互斥），不适合多标签场景；BCEWithLogitsLoss 使用 sigmoid，每个类别独立判断，适合多标签场景。

对于 MLLM 方案： 使用的是标准的**自回归交叉熵损失**，对生成的每个 token 计算交叉熵，只对标签部分的 token 计算损失（输入部分掩码为 -100）。

---

2. 你用的是什么优化器？为什么选择它?

使用 Adam 或 AdamW 优化器，和其他优化器区别看深度学习基础部分面经。

训练超参数

3. 基于图片的超参数？训练多久？什么卡?

参数	值
模型	ResNet18 / ViT-B/16
Batch Size	24
学习率	0.001
训练轮数	20 epochs
输入分辨率	224×224
采样帧数	16 帧/视频
优化器	Adam/AdamW
GPU	单卡 RTX 4090
训练时间	4h

---

4. 基于视频的超参数？训练多久？什么卡?

参数	值
模型	TimeSformer
学习率	0.001
Batch size	32
训练轮数	10 epochs
采样帧数	8 帧/视频
优化器	Adam/AdamW
GPU	单卡 RTX 4090

训练时间	3h

5. 基于超图的超参数？训练多久？什么卡?

参数	值
模型	ViT-H/14
Batch Size	24
学习率	0.001
训练轮数	20 epochs
输入分辨率	518×518
采样帧数	9 帧拼接为 3×3 超图
优化器	Adam/AdamW
GPU	单卡 4090
训练时间	4h

6. 基于 MLLM 的超参数？训练多久？什么卡?

参数	值
模型	Qwen2-VL-2B + LoRA
Batch Size	8
梯度累积步数	4（等效 batch size 32）
学习率	1e-4
训练轮数	5 epochs

LoRA 参数	r=8, alpha=32, dropout=0.01
输入分辨率	518×518
GPU	4090
训练时间	4h

五、评价指标相关

1. 请详细介绍 mAP 的计算过程?

mAP（mean Average Precision） 是多标签分类中常用的综合评价指标，计算过程如下：

第一步：针对单个类别计算 AP

以"腐蚀"类别为例：

1. **获取预测分数：** 模型为每个样本输出该类别的置信度分数（0 到 1 之间）。
2. **按分数排序：** 将所有样本按该类别的预测分数从高到低排序。
3. **逐步计算 P 和 R：** 从排序第 1 的样本开始，依次向下遍历：

- 在每个位置，计算 **Precision** = 到当前位置为止的真正样本数 / 到当前位置为止的总样本数 - 计算 **Recall** = 到当前位置为止的真正样本数 / 该类别的总真正样本数 - 例如，遍历到第 3 个样本时，如果其中有 2 个是真正的"腐蚀"样本，且总共有 5 个"腐蚀"样本，则 P=2/3，R=2/5

1. **绘制 PR 曲线：** 以 Recall 为横轴、Precision 为纵轴，将所有（R， P）点绘制为 PR 曲线。
2. **计算 AP：** AP 定义为 **PR 曲线下的面积**，反映了模型在不同召回水平下维持高精确率的能力。AP 取值范围[0， 1]，越大越好。

第二步：计算 mAP

对所有 C 个类别的 AP 取平均：

$$mAP = \frac{1}{C} \sum_{c=1}^{C} AP_c$$

例如，如果三个类别的 AP 分别为 1.00、0.83、0.90，则 mAP = （1.00 + 0.83 + 0.90） / 3 = 0.91。

mAP 的优势：

- 同时反映了预测的准确性（Precision）和检出的完整性（Recall）
- 不依赖于固定阈值，全面评估模型在各置信度水平下的表现
- 对每个类别独立评估后取平均，能反映模型在各类别上的均衡表现

六、优化和展望

1. 你觉得你的方案还有哪些可以改进的地方？

1. **长尾问题处理：** 可以类别平衡采样等专门的长尾学习策略，提升尾部类别的识别效果。
2. **更大规模的预训练模型：** 使用 VideoMAE、InternVideo 等在大规模视频数据上预训练的模型，可能获得更好的时空特征表示。
3. **MLLM 方案增强：** 使用更大参数量的多模态模型（如 7B）、更精细的 Prompt 工程、或者提取模型的 logits 概率用于更好的融合。

---

2. 项目中遇到过最大的困难是什么？怎么解决的？

困难一：长尾分布导致尾部类别性能差

尾部类别（如脱落、错口）的样本极少，模型几乎无法学到有效特征。

解决方案：

- 引入多模态大模型方案，利用大模型的预训练先验知识弥补数据不足
- 在 Prompt 中注入类别语义信息，帮助大模型理解少见的缺陷类型
- 多模型融合时，针对尾部类别给大模型方案更高的权重

困难二：多标签场景下标签噪声和模型混淆

视频帧方案中，将视频标签直接赋予每一帧，但很多帧可能只包含背景信息，引入了标签噪声。

解决方案：

- 采用超图方案，将多帧信息整合到一张图中，减少单帧的标签噪声影响
- 使用 BCEWithLogitsLoss 对每个类别独立建模，避免类别间的相互干扰

---

3. 未来感兴趣的研究方向

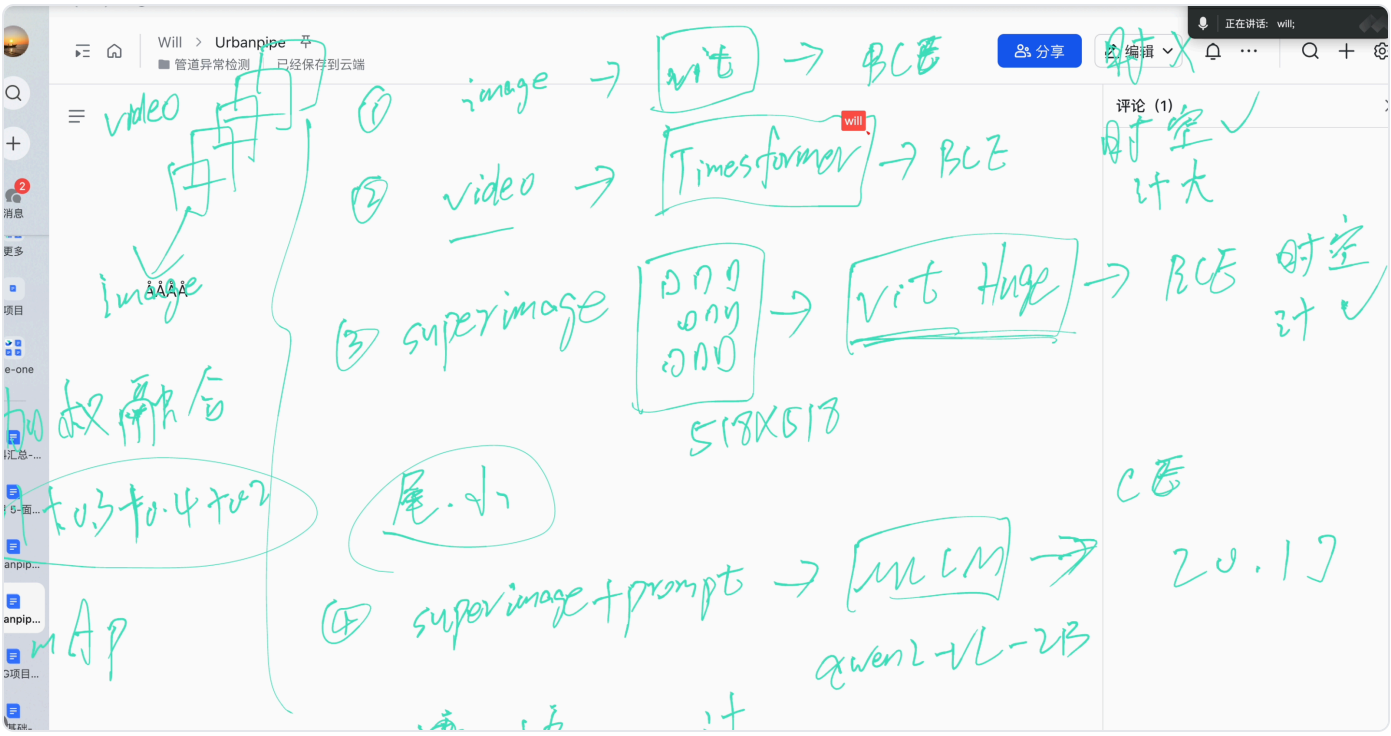
我未来的研究兴趣主要在**大模型（或者多模态）的持续学习与知识对齐**领域。

在参与‘城市管网缺陷检测’等项目过程中，我深刻体会到大模型在长效泛化上的优越性，尤其是在处理边缘案例和长尾分布数据时，展现出了传统小模型难以比拟的鲁棒性。

然而，我也观察到当前多模态大模型存在一些局限：即**‘离线训练的静态知识’与‘物理世界动态演进’之间的滞后性**。为了弥补这一鸿沟，我有一些了解：

1. **参数高效微调：**它的优势在于能让模型吸收特定领域的逻辑模式，实现**‘深度适配’**。但其局限在于对高质量数据的强依赖以及额外的训练开销。
2. **检索增强生成（RAG）：**通过引入外部实时知识库，解决大模型的幻觉问题，增强其在复杂工业场景下的决策可信度。但其局限在于需要额外的检索过程，会增加推理时的计算量与延迟。
3. **知识编辑（Knowledge Editing）：**如 AlphaEdit 这种‘外科手术式’的修正。它能在神经元级别实现**‘原子级更新’**而不破坏通用能力。虽然目前在大规模知识连续编辑上的效果仍有提升空间，但它提供了极高的更新效率。AlphaEdit: ICLR2025 Best Paper.

综上，我希望在研究生阶段围绕**如何让大模型更高效地获取、更新和利用知识**开展研究，并进一步探索**能够持续学习。**


Will > Urbanpipe
管道异常检测 已经保存到云端
分享
编辑
正在讲话: will;
评论 (1)
video
image
① image → ViT → BCE
② video → Timesformer → BCE
③ superimage → ViT Huge → BCE
superimage + prompt → MLLM
qwen2-VL-2B
BCE
20.17
时空 ✓
计大
时空
计 ✓
加权融合
0.3+0.4+0.2
map
尾.小
518X518